

6

MODELAGEM DE REQUISITOS: CENÁRIOS, INFORMAÇÕES E CLASSES DE ANÁLISE

CONCEITOS- -CHAVE

análise de domínio.	153
análise sintática ..	166
associações.....	176
casos de uso.....	157
classes de análise .	166
diagrama de	
atividades	161
diagrama de raios .	162
modelagem baseada	
em cenários	155
modelagem baseada	
em classes	166
modelagem de	
requisitos	154
modelagem CRC.....	171
modelagem e	
dados.....	163
modelos UML ..	161
pacotes de	
análise	178

No nível técnico, a engenharia de software começa com uma série de tarefas de modelagem que levam a especificação dos requisitos e representação do projeto para o software a ser construído. O modelo de requisitos¹ — na verdade, um conjunto de modelos — é a primeira representação técnica de um sistema.

Em um livro seminal sobre métodos de modelagem de requisitos, Tom DeMarco [DeM79] descreve o processo da seguinte maneira:

Revido reconhecidos problemas e falhas anteriores da fase de análise, sugiro que precisamos realizar os seguintes acréscimos ao nosso conjunto de metas. Os produtos da análise devem apresentar grande facilidade em sua manutenção. Isso se aplica particularmente ao Documento-Alvo [especificação de requisitos de software]. Problemas de tamanho devem ser tratados por meio de método efetivo de fracionamento. A especificação escrita como se fossem romances vitorianos está acabada. Elementos gráficos têm de ser usados sempre que possível. Temos de diferenciar as considerações lógicas [essenciais] das físicas [implementação]... No mínimo, precisamos... Algo que nos ajude a subdividir nossos requisitos e documentar essa subdivisão antes da especificação... Alguns meios de acompanhar e avaliar interfaces... Novas ferramentas para descrever lógica e política, algo melhor do que um texto narrativo.

Embora DeMarco tenha escrito sobre os atributos da modelagem de análise há mais de um quarto de século, seus comentários ainda se aplicam a métodos e notação da modelagem de requisitos moderna.

PANORAMA

O que é? A palavra escrita é um maravilhoso veículo para a comunicação, porém, não é necessariamente a melhor maneira de representar os requisitos de um software. A modelagem de requisitos usa uma combinação das formas textual e diagramática para representar os requisitos de maneira relativamente fácil de entender e, mais importante ainda, simples, para fazer a revisão em termos de correção, completude e consistência.

Quem realiza? Um engenheiro de software (às vezes chamado de “analista”) constrói o modelo usando os requisitos extraídos do cliente.

Por que é importante? Para validar os requisitos do software, é preciso examiná-los segundo uma série de pontos de vista. Neste capítulo consideraremos a modelagem de requisitos sob três perspectivas: modelos baseados em cenários, modelos de dados (informações) e modelos baseados em classes. Cada uma deles representa os requisitos em uma “dimensão” diferente, aumentando, portanto, a probabilidade de serem encontrados erros, inconsistências virem à tona e omissões serem reveladas.

Quais são as etapas envolvidas? A modelagem baseada em cenários representa o sistema sob o ponto de vista do usuário. A modelagem de dados representa o espaço de informações e os objetos de dados que o software irá manipular e os relacionamentos entre eles. A modelagem baseada em classes define objetos, atributos e relacionamentos. Assim que os modelos preliminares forem criados, eles são refinados e analisados para avaliar sua clareza, completude e consistência. No Capítulo 7, estendemos as dimensões da modelagem aqui citadas com representações adicionais, fornecendo uma visão mais robusta dos requisitos.

Qual é o artefato? Uma ampla gama de formas textuais e diagramáticas podem ser escolhidas para o modelo de requisitos. Cada uma das representações dá uma visão de um ou mais dos elementos de modelo.

Como garantir que o trabalho foi realizado corretamente? Os artefatos da modelagem de requisitos têm de ser revisados em termos de correção, completude e consistência. Devem refletir as necessidades de todos os interessados e estabelecer uma base a partir da qual o projeto pode ser conduzido.

¹ Em edições anteriores deste livro, usei o termo modelo de análise, em vez de modelo de requisitos. Nesta edição, decidi usar ambos os termos para representar a atividade de modelagem que define vários aspectos do problema a ser resolvido. Análise é a ação que ocorre à medida que requisitos são obtidos.

6.1 ANÁLISE DE REQUISITOS

A análise de requisitos resulta na especificação de características operacionais do software, indica a interface do software com outros elementos do sistema e estabelece restrições que o software deve atender. Permite ainda que você (independentemente de ser chamado de engenheiro de software, analista ou modelador) elabore mais as necessidades básicas estabelecidas durante as tarefas de concepção, levantamento e negociação, que são parte da engenharia de requisitos (Capítulo 5).

A ação da modelagem de requisitos resulta em um ou mais dos seguintes tipos de modelos:

- Modelos baseados em cenários de requisitos do ponto de vista de vários “atores” do sistema.
- Modelos de dados que representam o domínio de informações para o problema.
- Modelos orientados a classes que representam classes orientadas a objetos (atributos e operações) e a maneira por meio da qual as classes colaboram para atender aos requisitos do sistema.
- Modelos orientados a fluxos que representam os elementos funcionais do sistema e como eles transformam os dados à medida que percorrem o sistema.
- Modelos comportamentais que representam como o software se comporta em consequência de “eventos” externos.

Tais modelos dão a um projetista de software informações que podem ser traduzidas em projetos de arquitetura, de interfaces e de componentes. Finalmente, o modelo de requisitos (e a especificação de requisitos de software) fornecem ao desenvolvedor e ao cliente os meios para verificar a qualidade tão logo o software seja construído.

Neste capítulo, nos concentraremos na modelagem baseada em cenários — uma técnica que está se tornando cada vez mais popular na comunidade da engenharia de software; na modelagem de dados — técnica mais especializada que é particularmente apropriada quando uma aplicação tenha de criar ou manipular um espaço de informações complexo; e a modelagem de classes — uma representação de classes orientadas a objetos e as colaborações resultantes que permitem que um sistema funcione. Os modelos orientados a fluxos, os modelos comportamentais, a modelagem baseada em padrões e os modelos WebApp são discutidos no Capítulo 7.

“Qualquer ‘visão’ individual de requisitos é insuficiente para entender ou descrever o comportamento desejado de um sistema complexo.”

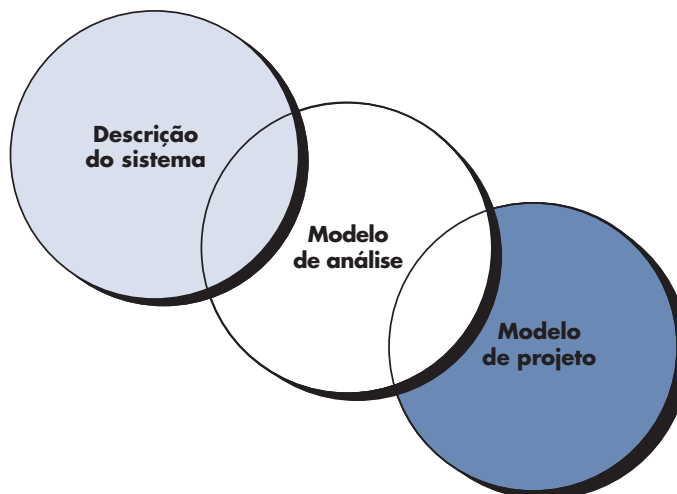
Alan M. Davis

PONTO-CHAVE

O modelo de análise e a especificação de requisitos fornecem um meio para avaliar a qualidade assim que o software for construído.

FIGURA 6.1

O modelo de requisitos como uma ponte entre a descrição do sistema e o modelo de projeto



“Requisito não é sinônimo de arquitetura. Requisito não é sinônimo de projeto nem de interface do usuário. Requisito é sinônimo de necessidade.”

Andrew Hunt e David Thomas

PONTO-CHAVE

O modelo de análise deve descrever o que o cliente quer, deve estabelecer uma base para o projeto, bem como definir uma meta para validação.

6.1.1 Filosofia e objetivos gerais

Na modelagem de requisitos, nosso foco primário é no que e não no como. Que interação com o usuário ocorre em dada circunstância, quais objetos o sistema manipula, que funções o sistema deve executar, quais comportamentos o sistema apresenta, que interfaces são definidas e quais restrições se aplicam?²

Em capítulos anteriores, citamos que a especificação de requisitos completa talvez não seja possível neste estágio. O cliente pode estar inseguro daquilo que é precisamente necessário para certos aspectos do sistema. O desenvolvedor pode estar incerto de que determinada abordagem irá cumprir apropriadamente as funções e o desempenho esperados. Essas realidades atenuam a favor de uma abordagem iterativa para a análise e modelagem de requisitos. O analista deve modelar aquilo que é conhecido e usar o modelo como base para o projeto do incremento de software.³

O modelo de requisitos deve alcançar três objetivos primários: (1) descrever o que o cliente solicita, (2) estabelecer uma base para a criação de um projeto de software e (3) definir um conjunto de requisitos que possa ser validado assim que o software for construído. O modelo de análise preenche a lacuna entre uma descrição sistêmica que descreve o sistema como um todo ou a funcionalidade de negócio que é atingida aplicando-se software, hardware, dados, pessoal e outros elementos de sistema e um projeto de software (Capítulos 8 a 13) que descreve a arquitetura, a interface do usuário e a estrutura em termos de componentes do software. Essa relação é ilustrada na Figura 6.1.

É importante notar que todos os elementos do modelo de requisitos estão diretamente associados a partes do modelo do projeto. Nem sempre é possível estabelecer uma clara divisão das tarefas de análise e de projeto entre essas duas importantes atividades de modelagem. Algum projeto ocorre invariavelmente como parte da análise e alguma análise será realizada durante o projeto.

6.1.2 Regras práticas para a análise

Arlow e Neustadt [Arl02] sugerem uma série de regras práticas que devem ser seguidas ao se criar o modelo de análise:

- *O modelo deve enfatizar as necessidades visíveis no domínio do problema ou do negócio. O nível de abstração deve ser relativamente elevado. “Não se perca nos detalhes”* [Arl02] que tentam explicar como o sistema irá funcionar.
- *Cada elemento do modelo de requisitos deve contribuir para o entendimento geral dos requisitos de software e fornecer uma visão do domínio de informação, função e comportamento do sistema.*
- *Postergue considerações de infraestrutura e outros modelos não funcionais até a fase de projeto.* Ou seja, talvez seja preciso um banco de dados, porém as classes necessárias para sua implementação, as funções necessárias para acessar o banco de dados e o comportamento que será apresentado à medida que ele for usado devem ser considerados apenas depois da análise do domínio do problema ter sido completada.
- *Minimize o acoplamento do sistema.* É importante representar os relacionamentos entre as classes e funções. Entretanto, se o nível de “interconexão” for extremamente alto, deve-se esforçar para reduzi-lo.
- *Certifique-se de que o modelo de requisitos agrega valor a todos os interessados. Cada participante tem um uso próprio para o modelo.* Por exemplo, os interessados no negócio devem usar o modelo para validar os requisitos; os projetistas devem usar o modelo como base para o projeto; o pessoal da Garantia da Qualidade (QA) deve usar o modelo para ajudar no planejamento de testes de aceitação.

 **Existem diretrizes básicas que podem nos ajudar enquanto realizamos atividades de análise de requisitos?**

“Problemas que valem a pena ser atacados demonstram seu valor contra-atacando.”

Piet Hein

2 Deve-se notar que à medida que os clientes se tornam tecnologicamente mais sofisticados, há uma tendência no sentido da especificação do *como* e também do *quê*. Entretanto, o foco principal deve permanecer no *quê*.
3 Como alternativa, a equipe de software poderia optar por criar um protótipo (Capítulo 2) em uma tentativa de entender melhor as necessidades do sistema.

- Mantenha o modelo o mais simples possível. Não crie diagramas adicionais quando não acrescentam nenhuma informação nova. Não utilize formas de notação complexas, quando uma lista simples já bastaria.

WebRef

Vários recursos úteis para a análise de domínio podem ser encontrados em www.iturils.com/English/SoftwareEngineering/SE_mod5.asp.

PONTO-CHAVE

A análise de domínio não examina uma aplicação específica, mas sim o domínio em que a aplicação reside. O intuito é identificar elementos comuns para resolução de problemas que sejam aplicáveis a todas as aplicações no domínio.

6.1.3 Análise de domínio

Na discussão da engenharia de requisitos (Capítulo 5), destaquei que frequentemente ocorre a recorrência de padrões de análise em muitas aplicações em um domínio de aplicação específico. Se esses padrões são definidos e categorizados de maneira que permita seu reconhecimento e sua aplicação para resolver problemas comuns, a criação do modelo de análise deve ser acelerada. Mais importante ainda, a probabilidade de aplicação de padrões de projeto e de componentes de software executáveis cresce dramaticamente. Isso melhora o tempo de colocação do produto no mercado e reduz custos de desenvolvimento.

Mas como os padrões e as classes de análise são inicialmente reconhecidos? Quem os define, os classifica e os prepara para uso em projetos subsequentes? As respostas a essas questões caem na análise de domínio. Firesmith [Fir93] descreve análise de domínio da seguinte maneira:

Análise de domínio de um software é a identificação, a análise e a especificação de requisitos comuns de um campo de aplicação específico, tipicamente para reutilização em vários projetos dentro deste campo de aplicação... [Análise de domínio orientada a objetos é] a identificação, análise e especificação de capacidades comuns reutilizáveis dentro de um campo de aplicação específico, em termos de objetos, classes, componentes e frameworks comuns.

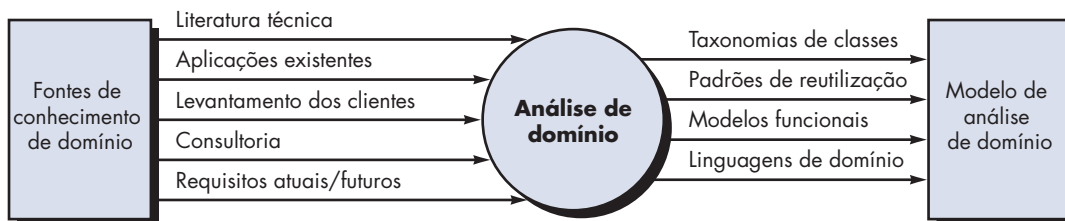
O “domínio de aplicação específico” pode ir da aviação a sistemas bancários, de videogames multimídia a software embutido em equipamentos médicos. O objeto da análise de domínio é simples: encontrar ou criar aquelas classes de análise e/ou padrões de análise largamente aplicáveis de modo que possam ser reutilizados.⁴

Usando a terminologia introduzida anteriormente neste livro, a análise de domínio pode ser vista como uma atividade universal para o processo de software. Queremos dizer com isso que a análise de domínio é uma atividade contínua da engenharia de software que não está ligada a nenhum projeto de software específico. De certo modo, o papel de um analista de domínio é similar ao papel de um mestre-ferramenteiro em um ambiente de manufatura pesada. O trabalho do mestre-ferramenteiro é projetar e construir ferramentas que possam ser usadas por muitas pessoas que fazem trabalhos semelhantes, mas não necessariamente iguais. O papel da análise de domínio⁵ é descobrir e definir padrões de análise, classes de análise e informações relacionadas que possam ser usados por várias pessoas que trabalham em aplicações similares, mas não necessariamente as mesmas.

A Figura 6.2 [Ara89] ilustra entradas e saídas fundamentais para o processo de análise de domínio. São consultadas fontes de conhecimento de domínio para identificar objetos que possam ser reutilizados no domínio.

FIGURA 6.2

Entrada e saída da análise de domínio



⁴ Uma visão complementar da análise de domínio “envolve a modelagem do domínio de forma que os engenheiros de software e outros interessados possam entender melhor sobre ela... Nem todas as classes de domínio resultam necessariamente no desenvolvimento de classes reutilizáveis...” [Let03a].

⁵ Não parta do pressuposto que pelo fato de um analista de domínio estar envolvido no trabalho, um engenheiro de software não precise entender o domínio de aplicação. Todos os membros da equipe de software precisam ter algum conhecimento do domínio em que o software será inserido.

CASA SEGURA

**Análise de domínio**

Cena: Sala do Doug Miller, após uma reunião com o pessoal de marketing.

Atores: Doug Miller, gerente de engenharia de software e Vinod Raman, membro da equipe de engenharia de software.

Conversa:

Doug: Preciso de você para um projeto especial, Vinod. Vou ter de tirá-lo das reuniões para levantamento de requisitos.

Vinod (franzindo a testa): Que pena. Aquele formato funciona realmente... Eu estava conseguindo extrair algo dessas reuniões. O que acontece?

Doug: Jamie e Ed te substituirão. De qualquer forma, o Marketing insiste que liberemos a capacidade de acesso à Internet com a função de segurança domiciliar na primeira versão do CasaSegura. Estamos com a corda no pescoço nesta... Não temos tempo ou gente suficiente; portanto, temos de resolver ambos os problemas — a interface PC e a interface Web — de uma só vez.

Vinod (parecendo confuso): Não sabia que o plano havia sido estabelecido... Nós ainda nem terminamos de fazer o levantamento de requisitos.

Doug (com um sorriso amarelo): Eu sei, mas os prazos são tão apertados que decidi começar a traçar uma estratégia com o Marketing agora mesmo... De qualquer maneira, revisaremos qualquer plano provisório assim que tivermos todas as informações obtidas nas reuniões de necessidades.

Vinod: OK, o que acontece? O que você quer que eu faça?

Doug: Você sabe o que é “análise de domínio”?

Vinod: Mais ou menos. Primeiramente, devo procurar padrões similares nas aplicações que fazem os mesmos tipos de coisas do que a que estou construindo. Se possível, você “rouba” então os padrões e os reutiliza em seu trabalho.

Doug: Não estou certo da palavra roubar, mas basicamente você entendeu a questão. O que eu gostaria que você fizesse é que começasse a pesquisar interfaces do usuário existentes de sistemas que controlam algo como o CasaSegura. Quero que você proponha um conjunto de padrões e classes de análise que possam ser comuns tanto para a interface baseada em PCs que ficarão instalados nos domicílios quanto a interface baseada em navegadores que pode ser acessada via Internet.

Vinod: Podemos poupar tempo fazendo com que eles sejam os mesmos... Por que não fazemos simplesmente isso?

Doug: Ah... É bom ter pessoas que pensem como você. Esse é o ponto — podemos poupar tempo e esforço se ambas as interfaces forem praticamente idênticas, implementadas com o mesmo código, blá, blá, blá, que o Marketing tanto insiste.

Vinod: Então, você quer o quê? Classes, padrões de análise, padrões de projeto?

Doug: Todos eles. Nada formal neste momento. Quero apenas ter alguma vantagem inicial em nossa análise interna e trabalho de projeto.

Vinod: Pesquisarei nossa biblioteca de classes e verei o que conseguimos. Também usarei um modelo de padrões que vi em um livro alguns meses atrás.

Doug: Muito bom. Mãos à obra!

“... análise é frustrante, cheia de complexos relacionamentos interpessoais, indefinida e difícil. Resumindo: é fascinante. Uma vez fisgado por ela, os antigos e fáceis prazeres da construção de um sistema jamais serão suficientes para satisfazê-lo novamente.”

Tom DeMarco

6.1.4 Abordagens à modelagem de requisitos

Uma visão da modelagem de requisitos, chamada *análise estruturada*, considera os dados e os processos que transformam os dados em entidades separadas. Os objetos de dados são modelados de maneira que definam seus atributos e relacionamentos. Processos que manipulam objetos de dados são modelados de maneira que mostrem como transformam dados à medida que os objetos de dados trafegam pelo sistema.

Uma segunda abordagem à modelagem de análise, denominada análise orientada a objetos, se concentra na definição de classes e na maneira pela qual colaboram entre si para atender às necessidades dos clientes. A UML e o Processo Unificado (Capítulo 2) são predominantemente orientados a objetos.

Embora o modelo de análise de requisitos proposto neste livro combine ambas as abordagens, as equipes de software em geral optam por uma abordagem e excluem todas as representações da outra. A questão não é qual é a melhor, mas sim, que combinação de representações dará aos interessados o melhor modelo de requisitos de software e a ponte mais efetiva com o projeto de software.

Cada elemento do modelo de requisitos (Figura 6.3) apresenta o problema segundo um ponto de vista. Os elementos baseados em cenários representam como o usuário interage com o sistema e a sequência específica de atividades que ocorre à medida que o software é

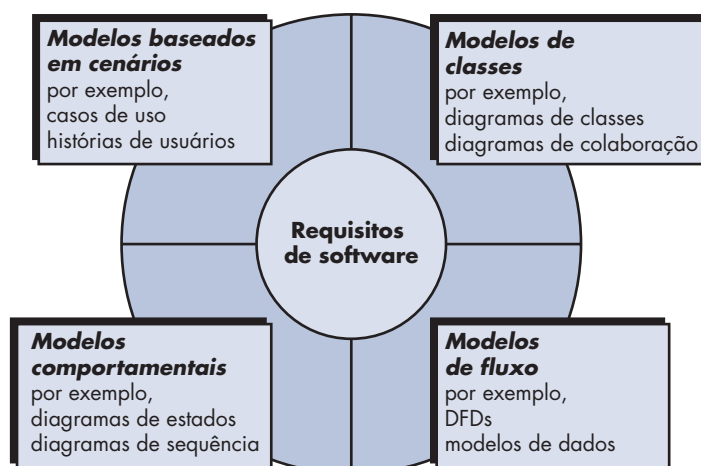
FIGURA 6.3

Elementos do modelo de análise

? Quais pontos de vista diferentes podem ser usados para descrever o modelo de requisitos?

“Por que devemos construir modelos? Por que não construir apenas o sistema? A resposta é que podemos construir modelos para destacar, ou enfatizar, certas características críticas de um sistema e, ao mesmo tempo, diminuir a importância de outros aspectos do sistema.”

Ed Yourdon



utilizado. Os elementos baseados em classes modelam os objetos que o sistema irá manipular, as operações que serão aplicadas aos objetos para efetuar a manipulação, os relacionamentos (alguns hierárquicos) entre os objetos e as colaborações que ocorrem entre as classes definidas. Os elementos comportamentais representam como eventos externos mudam o estado do sistema ou as classes que nele residem. Por fim, elementos orientados a fluxos representam o sistema como uma transformação de informações, indicando como os objetos de dados são transformados à medida que fluem pelas várias funções do sistema.

A modelagem de análise leva à obtenção de cada um desses elementos. Entretanto, o conteúdo específico de cada elemento (os diagramas usados para construir o elemento e o modelo) pode diferir de projeto para projeto. Como já citado várias vezes neste livro, a equipe de software tem de se esforçar para mantê-lo o mais simples possível. Devem ser utilizados apenas aqueles elementos da modelagem que agregam valor ao modelo.

6.2 MODELAGEM BASEADA EM CENÁRIOS

Embora o sucesso de um sistema ou produto baseado em computadores seja medido de muitas formas, a satisfação do usuário encontra-se no topo da lista. Se você entender como os usuários finais (e outros atores) querem interagir com um sistema, sua equipe de software estará mais capacitada a caracterizar, de maneira apropriada, os requisitos e a construir modelos de análise e projeto proveitosos.

Portanto, a modelagem de requisitos com UML⁶ começa com a criação de cenários na forma de casos de uso, diagramas de atividades e diagramas de raias.

6.2.1 Criação de um caso de uso preliminar

Alistair Cockburn caracteriza um caso de uso como um “contrato de comportamento” [Coc01b]. Conforme discutido no Capítulo 5, o “contrato” define a maneira através da qual um ator⁷ usa um sistema baseado em computadores para atingir alguma meta. Em essência, um caso de uso captura as interações que ocorrem entre produtores e consumidores de informação

“[Casos de uso] são apenas uma ferramenta para definir o que existe fora do sistema (atores) e o que deve ser realizado pelo sistema (casos de uso).”

Ivar Jacobson

⁶ A UML será usada como notação para modelagem ao longo deste livro. O Apêndice 1 apresenta um breve tutorial para aqueles que talvez não estejam familiarizados com a notação básica da UML.

⁷ Ator não é uma pessoa específica, mas sim um papel que uma pessoa (ou dispositivo) desempenha em um contexto específico. Um ator “invoca o sistema para que este realize um de seus serviços” [Coc01b].

e o sistema em si. Nesta seção, examinaremos como os casos de uso são desenvolvidos como parte da atividade da modelagem de requisitos.⁸



Em algumas situações, os casos de uso se tornam o mecanismo dominante de engenharia de requisitos. Entretanto, isso não significa que devemos descartar outros métodos de modelagem quando forem apropriados.

No Capítulo 5, citei que um caso de uso descreve um cenário de uso específico em uma linguagem simples sob o ponto de vista de um ator definido. Mas como saber: (1) sobre o que escrever, (2) quanto escrever a seu respeito, (3) com que nível de detalhamento fazer uma descrição e (4) como organizar a descrição? Essas são as questões que devem ser respondidas nas situações em que os casos de uso devam fornecer valor como uma ferramenta da modelagem de requisitos.

Sobre o que escrever? As duas primeiras tarefas da engenharia de requisitos — concepção e levantamento — nos fornecem informações necessárias para começarmos a escrever casos de uso. As reuniões para levantamento de requisitos, o QFD e outros mecanismos de engenharia de requisitos são utilizados para identificar interessados, definir o escopo do problema, especificar as metas operacionais globais, estabelecer as prioridades, descrever todos os requisitos funcionais conhecidos e descrever os itens (objetos) manipulados pelo sistema.

Para começar a desenvolver um conjunto de casos de uso, enumere as funções ou atividades realizadas por um ator específico. Podemos obtê-las de uma lista de funções dos requisitos do

CASA SEGURA



Desenvolvimento de outro cenário preliminar de usuário

Cena: Uma sala de reuniões, durante a segunda reunião para levantamento de requisitos.

Atores: Jamie Lazar, membro da equipe de software; Ed Robins, membro da equipe de software; Doug Miller, gerente da engenharia de software; três membros do Depto. de Marketing; um representante da Engenharia de Produto e um facilitador.

Conversa:

Facilitador: É hora de começarmos a falar sobre a função de vigilância do CasaSegura. Vamos desenvolver um cenário de usuário para acesso à função de vigilância.

Jamie: Quem desempenha o papel do ator nisto?

Facilitador: Acredito que a Meredith (uma pessoa do Marketing) venha trabalhando nessa funcionalidade. Por que você não desempenha o papel?

Meredith: Você quer fazer da mesma forma que fizemos da última vez, não é mesmo?

Facilitador: Correto... Da mesma forma.

Meredith: Bem, obviamente a razão para a vigilância é permitir ao proprietário do imóvel verificar a casa enquanto ele se encontra fora, gravar e reproduzir imagens de vídeo que são capturadas... Esse tipo de coisa.

Ed: Usaremos compressão para armazenamento de vídeo?

Facilitador: Boa pergunta, Ed, mas vamos postergar essas questões de implementação por enquanto. Meredith?

Meredith: Ok, então basicamente há duas partes para a função de vigilância... A primeira configura o sistema inclusive desenhando uma planta — precisamos de ferramentas para ajudar o proprietário do imóvel a fazer isto — e a segunda parte é a própria função de vigilância real. Como o layout faz parte da atividade de configuração, irei me concentrar na função de vigilância.

Facilitador (sorrindo): Você tirou as palavras de minha boca.

Meredith: Eu quero ter acesso à função de vigilância via PC ou via Internet. Meu sentimento é que o acesso via Internet seria mais utilizado. De qualquer maneira, quero poder exibir visões de câmeras em um PC e controlar o deslocamento e ampliação de imagens de determinada câmera. Especifico a câmera selecionando-a da planta da casa. Quero, de forma seletiva, gravar imagens geradas por câmeras e reproduzi-las. Também quero ser capaz de bloquear o acesso a uma ou mais câmeras com uma senha específica. Também quero a opção de ver pequenas janelas que mostrem visões de todas as câmeras e então poder escolher uma que deseje ampliar.

Jamie: Estas são chamadas de visões em miniatura.

Meredith: OK, então eu quero visões em miniatura de todas as câmeras. Também quero que a interface para a função de vigilância tenha o mesmo aspecto de todas as demais do CasaSegura. Quero que ela seja intuitiva, significando que não vou precisar ler todo o manual para usá-la.

Facilitador: Bom trabalho. Agora, vamos nos aprofundar um pouco mais nesta função...

⁸ Os casos de uso são uma parte importante da modelagem de análise para as interfaces do usuário. A análise de interfaces é discutida em detalhes no Capítulo 11.

sistema, por meio de conversações com interessados ou através de uma avaliação de diagramas de atividades (Seção 6.3.1) desenvolvidas como parte da modelagem de análise.

A função de vigilância domiciliar do CasaSegura (subsistema) discutida no quadro anterior identifica as seguintes funções (uma lista resumida) realizadas pelo ator **proprietário**:

- Selecionar câmera a ser vista.
- Solicitar imagens em miniatura de todas as câmeras.
- Exibir imagens das câmeras em uma janela de um PC.
- Controlar deslocamento e ampliação de uma câmera específica.
- Gravar, de forma seletiva, imagens geradas pelas câmeras.
- Reproduzir as imagens geradas pelas câmeras.
- Acessar a vigilância por câmeras via Internet.

À medida que as conversações com o interessado (que desempenha o papel de proprietário de um imóvel) forem avançando, a equipe de levantamento de requisitos desenvolve casos de uso para cada uma das funções citadas. Em geral, os casos de uso são escritos primeiro de forma narrativa informal. Caso seja necessária maior formalidade, o mesmo caso de uso é reescrito usando um formato estruturado similar àquele proposto no Capítulo 5 e reproduzido mais adiante, nesta seção, na forma de um quadro.

Para fins de ilustração, consideremos a função *acessar a vigilância por câmeras via Internet — exibir visões das câmeras* (**AVC-EVC**). O interessado que faz o papel do ator **proprietário** escreveria a seguinte narrativa:

Caso de uso: Acessar vigilância por câmeras via Internet — exibir visões das câmeras (AVC-EVC)

Ator: proprietário

Se eu estiver em um local distante, posso usar qualquer PC com navegador apropriado para entrar no site Produtos da CasaSegura. Introduzo meu ID de usuário e dois níveis de senhas e, depois de validado, tenho acesso a toda funcionalidade para o meu sistema CasaSegura instalado. Para acessar a visão de câmera específica, seleciono “vigilância” dos botões para as principais funções mostradas. Em seguida, seleciono “escolha uma câmera”, e a planta da casa é mostrada. Depois, seleciono a câmera em que estou interessado. Como alternativa, posso ver, simultaneamente, imagens em miniatura de todas as câmeras selecionando “todas as câmeras” como opção de visualização. Depois de escolher uma câmera, seleciono “visualização”, e uma visualização com um quadro por segundo aparece em uma janela de visualização identificada pelo ID de câmera. Se quiser trocar de câmeras, seleciono “escolha uma câmera”, e a janela de visualização original desaparece e a planta da casa é mostrada novamente. Em seguida, seleciono a câmera em que estou interessado. Surge uma nova janela de visualização.

Uma variação de um caso de uso narrativo apresenta a interação na forma de uma sequência ordenada de ações de usuário. Cada ação é representada como uma sentença declarativa. Voltando à função **AVC-EVC**, poderíamos escrever:

Caso de uso: Acessar vigilância por câmeras via Internet — exibir visões das câmeras (AVC-EVC)

Ator: proprietário

1. O proprietário do imóvel faz o login no site Produtos da CasaSegura.
2. O proprietário do imóvel introduz seu ID de usuário.
3. O proprietário do imóvel introduz duas senhas (cada uma com pelo menos oito caracteres).
4. O sistema mostra os botões de todas as principais funções.
5. O proprietário do imóvel seleciona a “vigilância” por meio dos botões das funções principais.
6. O proprietário do imóvel seleciona “escolher uma câmera”.
7. O sistema mostra a planta da casa.
8. O proprietário do imóvel seleciona um ícone de câmera da planta da casa.

“Os casos de uso podem ser utilizados em vários processos [de software]. Nosso processo favorito é o iterativo e dirigido por riscos.”

Geri Schneider e Jason Winters

9. O proprietário do imóvel seleciona o botão “visualização”.
10. O sistema mostra uma janela de visualização identificada pelo ID de câmera.
11. O sistema mostra imagens de vídeo na janela de visualização a uma velocidade de um quadro por segundo.

É importante notar que essa apresentação sequencial não leva em consideração quaisquer interações alternativas (a narrativa flui de forma mais natural e representa um número pequeno de alternativas). Casos de uso desse tipo são algumas vezes conhecidos como cenários primários [Sch98a].

6.2.2 Refinamento de um caso de uso preliminar

A descrição de interações alternativas é essencial para um completo entendimento da função a ser descrita por um caso de uso. Consequentemente, cada etapa no cenário primário é avaliado fazendo-se as seguintes perguntas [Sch98a]:

 Como examinar seqüências de ações alternativas ao desenvolver um caso de uso?

- *O ator pode fazer algo diferente neste ponto?*
- *Existe a possibilidade de o ator encontrar alguma condição de erro neste ponto? Em caso positivo, qual seria ela?*
- *Existe a possibilidade de o ator encontrar algum outro tipo de comportamento neste ponto (por exemplo, comportamento que é acionado por algum evento fora do controle do ator)? Em caso positivo, qual seria ele?*

Respostas a essas perguntas resultam na criação de um conjunto de *cenários secundários* que fazem parte do caso de uso original, mas representam comportamento alternativo. Consideremos, por exemplo, as etapas 6 e 7 do cenário primário apresentado anteriormente:

6. O proprietário do imóvel seleciona “escolher uma câmera”.
7. O sistema mostra a planta da casa.

Poderia o ator assumir outra atitude neste ponto? A resposta é “sim”. Referindo-se à narrativa que flui naturalmente, o ator poderia optar por ver, simultaneamente, imagens em miniatura de todas as câmeras. Portanto, um cenário secundário poderia ser “Visualizar imagens em miniatura para todas as câmeras.”

Existe a possibilidade de o ator encontrar alguma condição de erro neste ponto? Qualquer número de condições de erro pode ocorrer enquanto um sistema baseado em computadores opera. Nesse contexto, consideramos condições de erro apenas aquelas que provavelmente são resultado direto para realizar a ação nas etapas 6 ou 7. Enfatizando, a resposta à pergunta é “sim”. Uma planta com ícones de câmera talvez jamais tenha sido configurada. Portanto, selecionando “escolher uma câmera” resulta em condição de erro: “Não há nenhuma planta configurada para este imóvel”.⁹ Essa condição de erro se torna um cenário secundário.

Existe a possibilidade de o ator encontrar algum outro tipo de comportamento neste ponto? Mais uma vez a resposta é “sim”. Enquanto as etapas 6 e 7 ocorrem, o sistema pode encontrar uma condição de alarme. Isso resultaria no sistema exibir uma notificação de alerta especial (tipo, local, ação do sistema) e dar ao ator uma série de opções relevantes à natureza do alerta. Pelo fato de o cenário secundário poder ocorrer a qualquer momento para praticamente todas as interações, ele não fará parte do caso de uso **AVC-EVC**. Em vez disso, seria desenvolvido um caso de uso distinto — **Condição de alarme encontrada** — e referido de outros casos de uso conforme a necessidade.

Cada uma das situações descritas nos parágrafos anteriores é caracterizada como uma exceção do caso de uso. Uma *exceção* descreve uma situação (seja ela uma condição de falha, seja

⁹ Nesse caso, outro ator, o **administrador do sistema**, teria de configurar a planta da casa, instalar e inicializar (por exemplo, atribuir um ID de equipamento) todas as câmeras e testar cada uma delas para ter certeza de que ele se encontra acessível através do sistema e através da planta da casa.

uma alternativa escolhida pelo ator) que faz com que o sistema exiba um comportamento um tanto diferente.

Cockburn [Coc01b] recomenda o uso de uma sessão de *brainstorming* para obter um conjunto de exceções relativamente completo para cada caso de uso. Além das três perguntas genéricas sugeridas anteriormente nesta seção, as seguintes questões também deveriam ser exploradas:

- *Existem casos em que ocorre alguma “função de validação” durante este caso de uso?* Isso implica que a função de validação é chamada e poderia ocorrer uma condição de erro potencial.
- *Existem casos em que uma função de suporte (ou ator) parará de responder apropriadamente?* Por exemplo, uma ação de usuário aguarda uma resposta, porém a função que deve responder entra em condição de *time-out*.
- *Existe a possibilidade de um fraco desempenho do sistema resultar em ações do usuário inesperadas ou impróprias?* Por exemplo, uma interface baseada na Web responde de forma muito lenta, fazendo com que um usuário selecione um botão de processamento várias vezes seguidas. Essas seleções entram em fila de forma inapropriada e, por fim, geram condição de erro.

A lista de extensões desenvolvidas como consequência das perguntas e respostas deve ser “racionalizada” [Co01b] por meio dos seguintes critérios: deve ser indicada uma exceção no caso de uso se o software for capaz de detectar a condição descrita e, em seguida, tratar a condição assim que esta for detectada. Em alguns casos, uma exceção irá precipitar o desenvolvimento de um outro caso de uso (para tratar da condição percebida).

6.2.3 Criação de um caso de uso formal

Os casos de uso informais apresentados na Seção 6.2.1 são, algumas vezes, suficientes para a modelagem de requisitos. Entretanto, quando um caso de uso envolve uma atividade crítica ou descreve um conjunto complexo de etapas com um número significativo de exceções, uma abordagem mais formal talvez seja mais desejável.

O caso de uso **AVC-EVC** mostrado no quadro segue uma descrição geral típica para casos de uso formais. O *objetivo no contexto* identifica o escopo geral do caso de uso. A *precondição* descreve aquilo que é conhecido como verdadeiro antes de o caso de uso ser iniciado. O *disparador* identifica o evento ou a condição que “faz com que o caso de uso seja iniciado” [Coc01b]. O *cenário* enumera as ações específicas que o ator deve tomar e as respostas apropriadas do sistema. As *exceções* identificam as situações reveladas à medida que o caso de uso preliminar é refinado (Seção 6.2.2). Cabeçalhos adicionais poderão ou não ser acrescentados e são relativamente autoexplicativos.

WebRef

Quando termina a atividade de criação de casos de uso? Para uma proveitosa discussão sobre este tópico, veja ootips.org/usecases/done.html.

Em muitos casos, não há nenhuma necessidade de criar uma representação gráfica de um cenário de uso. Entretanto, a representação diagramática pode facilitar a compreensão, particularmente quando o cenário é complexo. Conforme já citado anteriormente neste livro, a UML oferece recursos de diagramação de casos de uso. A Figura 6.4 representa um diagrama de caso de uso preliminar para o produto *CasaSegura*. Cada caso de uso é representado por uma elipse. Nesta seção foi discutido apenas o caso de uso **AVC-EVC**.

Toda notação de modelagem tem suas limitações, e o caso de uso não é uma exceção. Assim como qualquer outra forma de descrição escrita, a qualidade de um caso de uso depende de seu(s) autor(es). Se a descrição não for clara, o caso de uso pode ser enganoso ou ambíguo. Um caso de uso que se concentra nos requisitos funcionais e comportamentais geralmente é inapropriado para requisitos não funcionais. Para situações em que o modelo de análise deve ter muitos detalhes e precisão (por exemplo, sistemas com segurança crítica), um caso de uso talvez não seja suficiente.

Entretanto, a modelagem baseada em cenários é apropriada para a grande maioria de todas as situações com as quais você irá deparar como engenheiro de software. Se desenvolvido apropriadamente, o caso de uso pode gerar grandes benefícios como ferramenta de modelagem.

CASA SEGURA



Modelo de caso de uso para vigilância

Caso de uso: Acessar a vigilância por câmeras via Internet — exibir visões das câmeras (AVC-EVC)

2, última modificação: 14 de janeiro feita por V. Raman.

Iteração:

Ator primário:

Proprietário do imóvel.

Objetivo no contexto:

Visualizar imagens de câmera espalhadas pela casa de qualquer ponto remoto via Internet.

Precondições:

O sistema deve estar totalmente configurado; devem ser obtidos ID de usuário e senhas apropriadas.

Disparador:

O proprietário do imóvel decide fazer uma inspeção na casa enquanto se encontra fora.

Cenário:

1. O proprietário do imóvel faz o login no site *Produtos da CasaSegura*.
2. O proprietário introduz seu ID de usuário.
3. O proprietário introduz duas senhas (cada uma com pelo menos oito caracteres).
4. O sistema mostra os botões de todas as principais funções.
5. O proprietário seleciona a "vigilância" por meio dos botões das funções principais.
6. O proprietário seleciona "escolher uma câmera".
7. O sistema mostra a planta da casa.
8. O proprietário seleciona um ícone de câmera da planta da casa.
9. O proprietário seleciona o botão "visualização".
10. O sistema mostra uma janela de visualização identificada pelo ID de câmera.
11. O sistema mostra imagens de vídeo na janela de visualização a uma velocidade de um quadro por segundo.

Exceções:

1. O ID ou senhas são incorretos ou não foram reconhecidos — veja o de uso **Validar ID e senhas**.
2. A função de vigilância não está configurada para este sistema — o sistema mostra a mensagem de erro apropriada; veja o caso de uso **Configurar função de vigilância**.
3. O proprietário seleciona "Visualizar as imagens em miniatura para todas as câmeras" — veja o caso de uso **Visualizar as imagens em miniatura para todas as câmeras**.
4. A planta não está disponível ou não foi configurada — exibir a mensagem de erro apropriada e ver o caso de uso **Configurar planta da casa**.
5. É encontrada uma condição de alarme — veja o caso de uso **Condição de alarme encontrada**.

Prioridade:

Prioridade moderada, a ser implementada após as funções básicas.

Quando disponível:

Terceiro incremento.

Frequência de uso:

Frequência moderada.

Canal com o ator:

Via navegador instalado em PC e conexão Internet.

Atores secundários:

Administrador do sistema, câmeras.

Canais com os atores secundários:

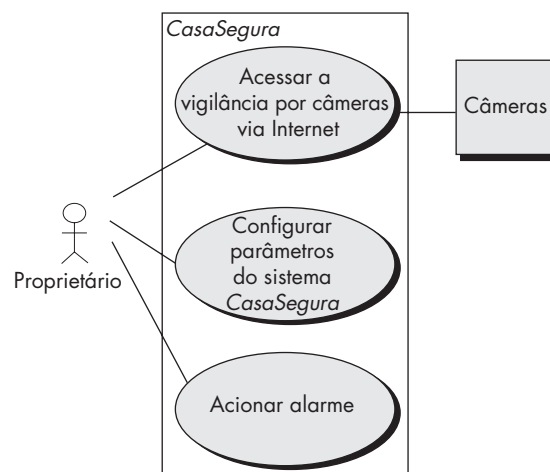
1. Administrador do sistema: sistema baseado em PCs.
2. Câmeras: conectividade sem fio.

Questões abertas:

1. Quais mecanismos protegem o uso não autorizado deste recurso por parte de funcionários da *Produtos da CasaSegura*?
2. A segurança é suficiente? Acessar de forma não autorizada este recurso representaria uma invasão de privacidade importante.
3. A resposta do sistema via Internet seria aceitável dada a largura de banda requerida para visualizações de câmeras?
4. Iremos desenvolver um recurso para fornecer vídeo a uma velocidade de quadros por segundo maior quando as conexões de banda larga estiverem disponíveis?

FIGURA 6.4

Diagrama de caso de uso preliminar para o sistema *CasaSegura*



6.3 MODELOS UML QUE COMPLEMENTAM O CASO DE USO

Existem muitas situações de modelagem de requisitos em que um modelo baseado em texto — mesmo um simples como o caso de uso — talvez não forneça informações de maneira clara e concisa. Em tais casos, pode-se optar por uma ampla gama de modelos gráficos da UML.

6.3.1 Desenvolvimento de um diagrama de atividade

Um diagrama de atividades UML complementa o caso de uso através de uma representação gráfica do fluxo de interação em um cenário específico. Similar ao fluxograma, um diagrama de atividades usa retângulos com cantos arredondados para representar determinada função do sistema, setas para representar o fluxo através do sistema, losangos de decisão para representar uma decisão com ramificação (cada seta saindo do losango é identificada) e as linhas horizontais cheias indicam as atividades paralelas que estão ocorrendo. Um diagrama de atividades para o caso de uso **AVC-EVC** é mostrado na Figura 6.5. Deve-se notar que o diagrama de atividades acrescenta outros detalhes não mencionados (mas implícitos) pelo caso de uso.

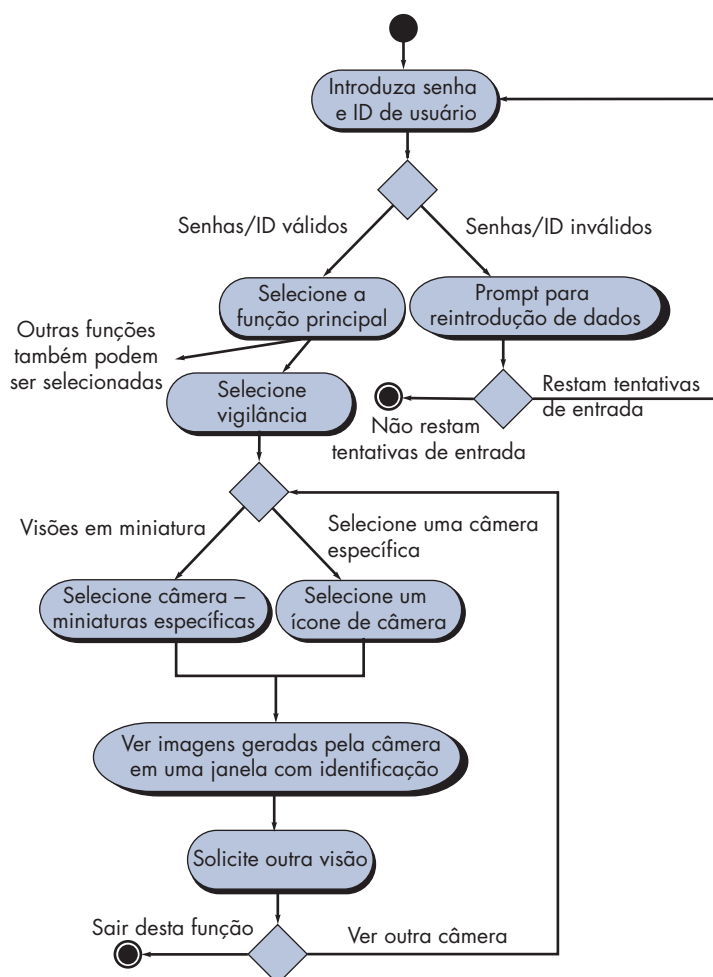
Por exemplo, um usuário poderia tentar introduzir **ID de usuário** e **senha** um número limitado de vezes. Isso é representado pelo losango de decisão abaixo de “Prompt para reintrodução de dados”.

PONTO-CHAVE

Um diagrama de atividades UML representa as ações e decisões que ocorrem enquanto uma dada função é executada.

FIGURA 6.5

Diagrama de atividades para a função **Acessar a vigilância por câmeras via Internet** — exibir visões das câmeras



PONTO-CHAVE

Um diagrama de raias UML representa o fluxo de ações e decisões e indica quais atores realizam cada uma delas.

“Um bom modelo orienta nossas ideias, enquanto um ruim as distorce.”

Brian Marick

6.3.2 Diagramas de raias

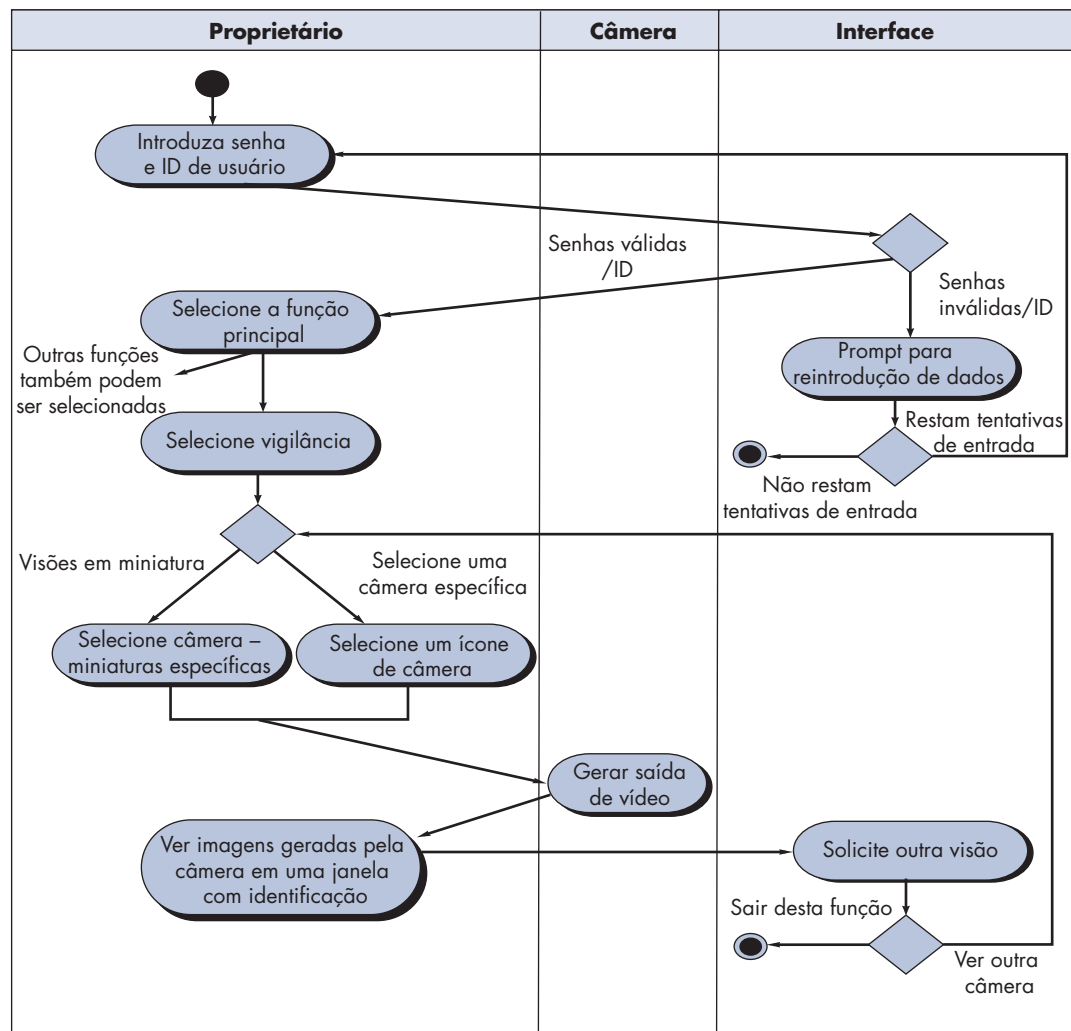
O diagrama de raias UML é uma útil variação do diagrama de atividades e nos permite representar o fluxo de atividades descrito pelo caso de uso e, ao mesmo tempo, indicar qual ator (se existirem vários atores envolvidos em determinado caso de uso) ou classe de análise (discutida posteriormente neste capítulo) tem a responsabilidade pela ação descrita por um retângulo de atividade. As responsabilidades são representadas como segmentos paralelos que dividem o diagrama verticalmente, como as raias de uma piscina.

Três classes de análise — **Proprietário, Câmera e Interface** — possuem responsabilidades diretas ou indiretas no contexto do diagrama de atividades representado na Figura 6.5.

Referindo-se à Figura 6.6, o diagrama de atividades é rearranjado de forma que as atividades associadas a determinada classe de análise caiam na raia para a referida classe. Por exemplo, a classe **Interface** representa a interface do usuário conforme vista pelo proprietário. O diagrama de atividades indica dois *prompts* que são a responsabilidade da interface — “prompt para reintrodução de dados” e “prompt para outra visão”. Esses prompts e as decisões associadas a eles caem na raia **Interface**. Entretanto, as setas saem dessa raia e voltam para a raia do **Proprietário**, onde ocorrem as ações do proprietário do imóvel.

FIGURA 6.6

Diagrama de raias para a função **Acessar a vigilância por câmeras via Internet** — exibir visões das câmeras



Casos de uso, juntamente com os diagramas de atividades e de raias, são orientados a procedimentos. Eles representam a maneira pela qual vários atores chamam funções específicas (ou outras etapas procedurais) para atender aos requisitos do sistema. Porém, uma visão procedural dos requisitos representa apenas uma única dimensão de um sistema. Na Seção 6.4, examinamos o espaço de informações e como os requisitos de dados podem ser representados.

6.4 CONCEITOS DE MODELAGEM DE DADOS

WebRef

Informações úteis sobre a modelagem de dados podem ser encontradas em www.data-model.org.

? Como um objeto de dados se manifesta no contexto de uma aplicação?

PONTO-CHAVE

Objeto de dados é uma representação de qualquer informação composta que é processada pelo software.

PONTO-CHAVE

Os atributos dão nome a um objeto de dados, descrevem suas características e, em alguns casos, fazem referência a um outro objeto.

Se entre os requisitos de software tivermos a necessidade de criar, estender ou interfacear com um banco de dados ou se as estruturas de dados complexas tiverem de ser construídas e manipuladas, a equipe de software poderá optar por criar um *modelo de dados* como parte da modelagem de requisitos. Um analista ou engenheiro de software define todos os objetos de dados processados no sistema, os relacionamentos entre os objetos de dados e outras informações pertinentes aos relacionamentos. O *diagrama entidade-relacionamento* (*entity-relationship diagram*) trata das questões e representa todos os objetos de dados introduzidos, armazenados, transformados e produzidos em uma aplicação.

6.4.1 Objetos de dados

Objeto de dados é uma representação das informações compostas que devem ser compreendidas pelo software. Por *informação composta*, queremos dizer algo que tenha uma série de propriedades ou atributos diferentes. Consequentemente, a largura (um único valor) não seria um objeto de dados válido, mas **dimensões** (incorporando altura, largura e profundidade) poderia ser definido como um objeto.

Um objeto de dados pode ser uma entidade externa (por exemplo, qualquer item que produza ou consuma informações), uma coisa (por exemplo, um relatório ou uma exibição), uma ocorrência (por exemplo, uma ligação telefônica) ou evento (por exemplo, um alerta), um papel (por exemplo, vendedor), uma unidade organizacional (por exemplo, departamento de contabilidade), um local (por exemplo, um armazém) ou uma estrutura (por exemplo, um arquivo). Por exemplo, uma **pessoa** ou um **carro** podem ser vistos como objetos de dados no sentido de que ambos podem ser definidos em termos de um conjunto de atributos. A descrição do objeto de dados incorpora o objeto de dados e todos os seus atributos.

Um objeto de dados encapsula apenas dados — não há nenhuma referência em um objeto de dados a operações que atuam sobre os dados.¹⁰ Consequentemente, o objeto de dados pode ser representado como uma tabela conforme mostrado na Figura 6.7. Os cabeçalhos da tabela refletem os atributos do objeto. Nesse caso, um carro é definido em termos de sua **marca, modelo, número do chassi, tipo de carroceria, cor e proprietário**. O corpo da tabela representa instâncias específicas do objeto de dados. Por exemplo, um Chevy Corvette é uma instância do objeto de dados **carro**.

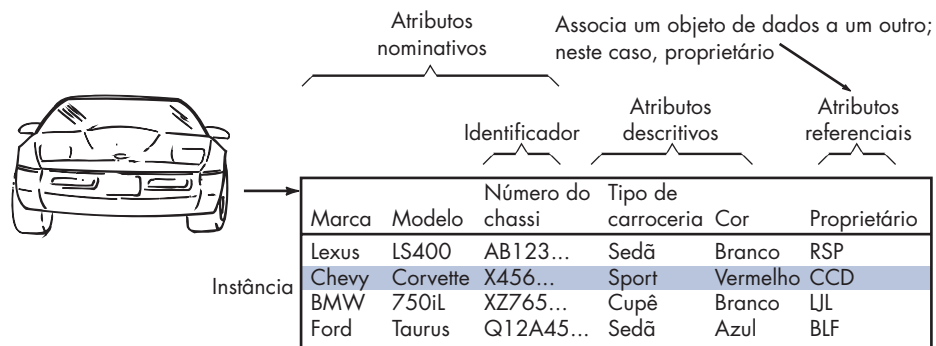
6.4.2 Atributos de dados

Os *atributos de dados* definem as propriedades de um objeto de dados e assumem uma de três características diferentes. Eles podem ser usados para (1) dar um nome a uma instância do objeto de dados, (2) descrever a instância ou (3) fazer referência a uma outra instância em outra tabela. Além disso, um ou mais dos atributos devem ser definidos como identificador — isto é, o atributo identificador se torna uma “chave” quando queremos encontrar uma instância do objeto de dados. Em alguns casos, os valores para o(s) identificador(es) são exclusivos, embora não seja uma exigência. Fazendo referência ao objeto de dados **carro**, um identificador razoável poderia ser o **número do chassi**.

¹⁰ Essa distinção separa o objeto de dados da classe ou objeto definido como parte da abordagem orientada a objetos (Apêndice 2).

FIGURA 6.7

Representação tabular de objetos de dados



WebRef

Um conceito chamado “normalização” é importante para aqueles que pretendem realizar uma modelagem de dados completa. Uma útil introdução pode ser encontrada em www.datamodel.org.

O conjunto de atributos apropriado para um objeto de dados é determinado por meio do entendimento do contexto do problema. Os atributos para **carro** poderiam servir bem para uma aplicação a ser usada por um departamento de veículos a motor, porém tais atributos seriam de pouca valia para uma empresa do setor automobilístico que precisasse fabricar software de controle. Neste último caso, os atributos para **carro** também poderiam incluir o **número do chassi**, o **tipo de carroceria** e a **cor**, porém, muitos outros atributos (por exemplo, **código interno**, **tipo de acionamento do motor**, **indicador do conjunto de tapeçaria**, **tipo de transmissão**) teriam de ser acrescentados para tornar **carro** um objeto com significado no contexto de controle de manufatura.

INFORMAÇÕES



Objetos de dados e classes orientadas a objetos — são eles a mesma coisa?

Uma pergunta que comumente surge quando o tema é objetos de dados: Um objeto de dados é a mesma coisa que uma classe orientada a objetos¹¹? A resposta é “não”.

O objeto de dados define um dado composto; ou seja, ele incorpora um conjunto de dados individuais (atributos) e dá um nome ao conjunto de itens (o nome do objeto de dados).

Uma classe orientada a objetos encapsula atributos de dados, mas também incorpora as operações (métodos) que manipulam os dados decorrentes desses atributos. Além disso, a definição de classes implica uma ampla infraestrutura que faz parte da abordagem de engenharia de software orientada a objetos. As classes se comunicam entre si via mensagens, podem ser organizadas em hierarquias e fornecem características de herança para objetos que são uma instância de uma classe.

6.4.3 Relacionamentos

Os objetos de dados são interligados de várias formas. Consideremos os dois objetos de dados, **pessoa** e **carro**. Estes podem ser representados usando-se a notação simples ilustrada na Figura 6.8a. É estabelecida uma conexão entre **pessoa** e **carro**, pois os dois objetos estão interrelacionados. Mas quais são os relacionamentos? Para determinarmos a resposta, devemos entender o papel das pessoas (proprietários, neste caso) e dos carros no contexto do software a ser construído. Podemos estabelecer um conjunto de pares objeto/relacionamento que definem os relacionamentos relevantes. Por exemplo,

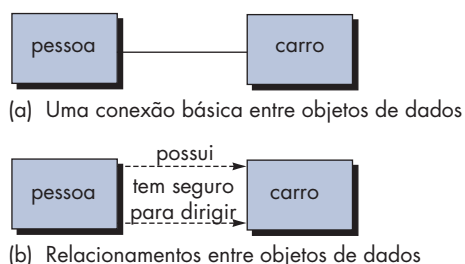
- Uma pessoa *possui* um carro.
- Uma pessoa *tem seguro para dirigir* um carro.

PONTO-CHAVE

Os relacionamentos indicam a maneira por meio da qual os objetos de dados são interligados entre si.

¹¹ Os leitores que não estiverem familiarizados com os conceitos e a terminologia da orientação a objetos devem consultar o breve tutorial apresentado no Apêndice 2.

FIGURA 6.8

Relacionamentos entre objetos de dados

Os relacionamentos *possui* e *tem seguro para dirigir* definem as conexões relevantes entre **pessoa** e **carro**. A Figura 6.8b ilustra graficamente esses pares objeto-relacionamento. As setas indicadas na Figura 6.8b dão uma importante informação sobre a direção do relacionamento e, normalmente, reduzem a ambiguidade ou interpretações incorretas.

**Diagramas entidade-relacionamento**

O par objeto-relacionamento é a pedra angular do modelo de dados. Os pares podem ser representados graficamente por meio do diagrama entidade-relacionamento (DER).¹² O DER foi originalmente proposto por Peter Chen [Che77] para o projeto de sistemas de bancos de dados relacionais e foi estendido por outros. Um conjunto de componentes primordiais é identificado para o DER: objetos de dados, atributos, relacionamentos e indicadores de vários tipos. O principal objetivo do DER é representar objetos de dados e seus relacionamentos.

Já foi feita uma introdução rudimentar sobre a notação do DER. Os objetos de dados são representados por um retângulo identificado. Os relacionamentos são indicados com uma linha rotulada interligando objetos. Em algumas variantes do DER, a linha de conexão contém um losango, rotulado com um relacionamento. As conexões entre objetos de dados e relacionamentos são estabelecidas usando-se uma variedade de símbolos especiais que indicam a cardinalidade e a modalidade.¹³ Se desejar maiores informações sobre modelagem de dados e o diagrama entidade-relacionamento, consulte [Hob06] ou [Sim05].

INFORMAÇÕES**Modelagem de dados**

Objetivo: As ferramentas de modelagem de dados dão a um engenheiro de software a capacidade de representar objetos de dados, suas características e seus relacionamentos. Usado primariamente para aplicações de bancos de dados de grande porte e outros projetos de sistemas de informação, as ferramentas para modelagem de dados fornecem um meio automatizado para criar diagramas entidade-relacionamento completos, dicionários de objetos de dados e modelos relacionados.

Mecânica: As ferramentas nesta categoria permitem ao usuário descrever objetos de dados e seus relacionamentos. Em alguns casos, as ferramentas usam a notação DER. Em outros, as ferramentas modelam relações por meio de algum outro mecanismo. As ferramentas nesta categoria são em geral usadas como parte do projeto de um banco de dados e permitem a criação de um modelo de banco de dados através da geração

FERRAMENTAS DO SOFTWARE

de um esquema de banco de dados para sistemas de gerenciamento de bancos de dados comuns (SGBD).

Ferramentas representativas¹⁴:

AllFusion ERWin, desenvolvida pela Computer Associates (www.3.ca.com), auxilia no projeto de objetos de dados, na estrutura apropriada e elementos-chave para bancos de dados.

ER/Studio, desenvolvida pela Embarcadero Software (www.embarcadero.com), oferece suporte à modelagem entidade-relacionamento.

Oracle Designer, desenvolvida pela Oracle Systems (www.oracle.com), “modela processos de negócio, entidades de dados e relacionamentos [que] são transformados em projetos a partir dos quais são gerados aplicações e bancos de dados completos”.

Visible Analyst, desenvolvida pela Visible Systems (www.visible.com), oferece suporte a uma série de funções de modelagem de análise inclusive à modelagem de dados.

12 Embora o DER ainda seja utilizado em algumas aplicações de projeto de bancos de dados, a notação UML (Apêndice 1) agora pode ser usada para projeto de dados.

13 *Cardinalidade* de um par objeto-relacionamento especifica “o número de ocorrências de um [objeto] que pode ser relacionado ao número de ocorrências de um outro [objeto]” [Til93]. A *modalidade* de um relacionamento é 0 se não houver uma necessidade explícita para a ocorrência do relacionamento ou o relacionamento for opcional. A modalidade é 1 se a ocorrência do relacionamento for compulsória.

14 As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra de ferramentas nesta categoria. Na maioria dos casos, os nomes das ferramentas são marcas registradas por seus respectivos desenvolvedores.

6.5 MODELAGEM BASEADA EM CLASSES

A modelagem baseado em classes representa os objetos que o sistema irá manipular, as operações (também denominadas métodos ou serviços) que serão aplicadas aos objetos para efetuar a manipulação, os relacionamentos (alguns hierárquicos) entre os objetos e as colaborações que ocorrem entre as classes definidas. Os elementos de um modelo baseado em classes são: classes e objetos, atributos, operações, modelos CRC (classe-responsabilidade-colaborador), diagramas de colaboração e pacotes. As seções a seguir apresentam uma série de diretrizes informais que auxiliarão na identificação e na representação desses elementos.

6.5.1 Identificação de classes de análise

Se você observar em torno de uma sala, existe um conjunto de objetos físicos que podem ser facilmente identificados, classificados e definidos (em termos de atributos e operações). Porém, quando se “observa em torno” do espaço de problema de uma aplicação de software, as classes (e objetos) podem ser mais difíceis de compreender.

Podemos começar a identificar classes examinando os cenários de uso desenvolvidos como parte do modelo de requisitos e realizando uma “análise sintática” [Abb83] dos casos de uso desenvolvidos para o sistema a ser construído. As classes são determinadas sublinhando-se cada substantivo ou frase contendo substantivos e introduzindo-a em uma tabela simples. Podem ser observados sinônimos. Se for preciso que a classe (substantivo) implemente uma solução, então ela faz parte do espaço de soluções; caso contrário, se for necessária uma classe apenas para descrever uma solução, ela faz parte do espaço de problemas.

Mas o que devemos procurar uma vez que todos os substantivos tenham sido isolados? As *classes de análise* se manifestam em uma das seguintes maneiras:

- *Entidades externas* (por exemplo, outros sistemas, dispositivos, pessoas) que produzem ou consomem informações a ser usadas por um sistema baseado em computadores.
- *Coisas* (por exemplo, relatórios, exibições, letras, sinais) que fazem parte do domínio de informações para o problema.
- *Ocorrências ou eventos* (por exemplo, uma transferência de propriedades ou a finalização de uma série de movimentos de robô) que ocorrem no contexto da operação do sistema.
- *Papéis* (por exemplo, gerente, engenheiro, vendedor) desempenhados pelas pessoas que interagem com o sistema.
- *Unidades organizacionais* (por exemplo, divisão, grupo, equipe) que são relevantes para uma aplicação.
- *Locais* (por exemplo, chão de fábrica ou área de carga) que estabelecem o contexto do problema e a função global do sistema.
- *Estruturas* (por exemplo, sensores, veículos de quatro rodas ou computadores) que definem uma classe de objetos ou classes de objetos relacionadas.

Essa categorização é apenas uma das muitas que foram propostas na literatura.¹⁵ Por exemplo, Budd [Bud96] sugere uma taxonomia de classes que inclui *produtores* (fontes) e *consumidores* (reservatórios) de dados, *gerenciadores de dados*, *classes de visualização ou de observação* e *classes auxiliares*.

Também é importante notar o que as classes ou objetos não devem ser. Em geral, uma classe jamais deve ter um “nome procedural imperativo” [Cas89]. Por exemplo, se os desenvolvedores de software de um sistema de imagens para aplicação em medicina definiram um objeto com o nome **InverterImagem** ou até mesmo **InversãoDeImagem**, eles estariam cometendo um erro sutil. A **Imagem** obtida do software poderia, obviamente, ser uma classe (é algo que faz parte do domínio de informações). A inversão da imagem é uma operação aplicada ao objeto.

“O problema realmente difícil é descobrir, logo no início, quais são os objetos [classes] corretos.”

Carl Argila

 De que maneira as classes de análise se manifestam como elementos do espaço de soluções?

15 Outra classificação importante, definindo classes de entidades, de contorno e de controle, é discutida na Seção 6.5.4.

É provável que a inversão seja definida como uma operação para o objeto **Imagem**, mas não seja definida como uma classe separada com a conotação “inversão de imagem”. Como afirma Cashman [Cas89]: “o intuito da orientação a objetos é encapsular, mas, ainda, manter separados os dados e as operações sobre os dados”.

Para ilustrarmos como as classes de análise poderiam ser definidas durante os estágios iniciais da modelagem, consideremos uma análise sintática (os substantivos são sublinhados, os verbos colocados em *itálico*) para uma narrativa¹⁶ de processamento da *função de segurança domiciliar do CasaSegura*.



A análise sintática não é infalível, mas um excelente salto inicial, caso você esteja em dificuldades para definir objetos de dados e as transformações que operam sobre eles.

A função de segurança domiciliar do CasaSegura *permite* que o proprietário do imóvel *configure* o sistema de segurança quando ele é *instalado*, *monitore* todos os sensores conectados ao sistema de segurança e *interaja* com o proprietário do imóvel através da Internet, de um PC ou de um painel de controle.

Durante a instalação, o PC do CasaSegura é usado para *programar* e *configurar* o sistema. É atribuído um número e um tipo a cada sensor, é programada uma senha-mestra para *armar* e *desarmar* o sistema e são *introduzidos* número(s) de telefone para *discar* quando um evento de sensor ocorre.

Quando um evento de sensor é *reconhecido*, o software *aciona* um alarme audível agregado ao sistema. Após um tempo de retardo que é *especificado* pelo proprietário do imóvel durante as atividades de configuração do sistema, o software *disca* um número de telefone de um serviço de monitoramento, *fornece* informações sobre o local, *relatando* a natureza do evento que foi detectado. O número de telefone será *discado novamente* a cada 20 segundos até que seja *completada* a ligação.

O proprietário do imóvel *recebe* informações de segurança através de um painel de controle, do PC ou de um navegador, coletivamente denominados interface. A interface *mostra* mensagens ao operador, bem como informações sobre o estado do sistema no painel de controle, no PC ou na janela do navegador. A interação com o proprietário do imóvel acontece da seguinte forma...

Extraindo os substantivos, podemos propor uma série de possíveis classes:

Classe potencial	Classificação geral
proprietário do imóvel	papel ou entidade externa
sensor	entidade externa
painel de controle	entidade externa
instalação	ocorrência
sistema (também conhecido como sistema de segurança)	coisa
número, tipo,	não objetos, atributos do sensor
senha-mestra	coisa
número de telefone	coisa
evento de sensor	ocorrência
alarme audível	entidade externa
serviço de monitoramento	unidade organizacional ou entidade externa

Prosseguiríamos com a lista até que todos os substantivos contidos na narrativa de processamento tivessem sido considerados. Observe que chamamos cada entrada da lista de possível objeto. Temos de considerar cada um deles até que uma decisão final seja feita.

¹⁶ Uma narrativa de processamento é similar ao caso de uso em termos de estilo, mas ligeiramente diferente em seu propósito. A narrativa de processamento fornece uma descrição geral da função a ser desenvolvida. Ela não é um cenário escrito sob o ponto de vista de um ator. É importante observar, entretanto, que uma análise sintática também pode ser usada para todos os casos de uso desenvolvidos como parte do levantamento de requisitos (inferência).

? Como podemos determinar se uma classe potencial deve, de fato, se tornar uma classe de análise?

Coad e Yourdon [Coa91] sugerem seis características de seleção que deveriam ser usadas à medida que se considera cada classe potencial para inclusão no modelo de análise:

1. *Informações retidas.* A classe potencial será útil durante a análise apenas se as informações sobre ela tiverem de ser lembradas para que o sistema possa funcionar.
2. *Serviços necessários.* A classe potencial deve ter um conjunto de operações identificáveis capazes de modificar, de alguma forma, o valor de seus atributos.
3. *Atributos múltiplos.* Durante a análise de requisitos, o foco deve ser nas informações “importantes”; uma classe com um único atributo poderia, na verdade, ser útil durante o projeto, porém, provavelmente seria mais bem representada na forma de atributo de outra classe durante a atividade de análise.
4. *Atributos comuns.* Um conjunto de atributos pode ser definido para a classe potencial e esses atributos se aplicam a todas as instâncias da classe.
5. *Operações comuns.* Um conjunto de operações pode ser definido para a classe potencial e tais operações se aplicam a todas as instâncias da classe.
6. *Requisitos essenciais.* Entidades externas que aparecem no espaço do problema e produzem ou consomem informações essenciais à operação de qualquer solução para o sistema quase sempre serão definidas como classes no modelo de requisitos.

“As classes batalham, algumas delas triunfam, outras são eliminadas.”

Mao Zedong

Para ser considerada uma classe legítima para inclusão no modelo de requisitos, um objeto potencial deve satisfazer todas (ou quase todas) essas características. A decisão para inclusão de classes potenciais no modelo de análise é um tanto subjetiva, e uma avaliação posterior poderia fazer com que um objeto fosse descartado ou reintegrado. Entretanto, a primeira etapa da modelagem baseada em classes é definir as classes e decisões (mesmo aquelas subjetivas). Com isso em mente, devemos aplicar as características de seleção da lista de classes potenciais da *CasaSegura*:

Classe potencial	Número característico que se aplica
proprietário do imóvel	rejeitado: 1, 2 falham, embora 6 se aplique
sensor	aceito: todos se aplicam
painel de controle	aceito: todos se aplicam
instalação	rejeitado
sistema (também conhecido como sistema de segurança)	aceito: todos se aplicam
número, tipo	rejeitado: 3 falha, atributos de sensor
senha-mestra	rejeitado: 3 falha
número de telefone	rejeitado: 3 falha
evento de sensor	aceito: todos se aplicam
alarme audível	aceito: 2, 3, 4, 5, 6 se aplicam
serviço de monitoramento	rejeitado: 1, 2 falham, embora 6 se aplique

Deve-se notar o seguinte: (1) a lista anterior não é definitiva, outras classes talvez tenham de ser acrescentadas para completar o modelo; (2) algumas das classes potenciais rejeitadas se tornarão atributos para as que foram aceitas (por exemplo, número e tipo são atributos de **Sensor** e senha-mestre e número de telefone talvez se tornem atributos de **Sistema**); (3) enunciados do problema diferentes talvez provoquem decisões “aceito ou rejeitado” diferentes (por exemplo, se cada proprietário de um imóvel tivesse uma senha individual ou fosse identificado pelo seu padrão de voz, a classe **Proprietário** satisfaria as características 1 e 2 e teriam sido aceitas).

PONTO-CHAVE

Atributos são o conjunto de objetos de dados que definem completamente a classe no contexto do problema.

6.5.2 Especificação de atributos

Os *atributos* descrevem uma classe selecionada para ser incluída no modelo de requisitos. Em essência, são os atributos que definem uma classe — que esclarecem o que a classe representa no contexto do espaço de problemas. Por exemplo, se fôssemos construir um sistema que registrasse estatísticas dos jogadores do campeonato de beisebol profissional, os atributos da classe **Jogador** seriam bem diferentes dos atributos da mesma classe quando usados no contexto do sistema de aposentadoria dos jogadores profissionais de beisebol. No primeiro, atributos como **nome**, **posição**, **média de rebatidas**, **porcentagem de voltas completas em torno do campo**, **número de anos jogados** e **número de jogos** podem ser relevantes. No outro caso, alguns desses atributos seriam relevantes, mas outros substituídos (ou aumentados) por atributos como **salário médio**, **crédito faltante para aposentadoria plena**, **opções escolhidas para o plano de aposentadoria**, **endereço para correspondência** e outros similares.

Para criarmos um conjunto de atributos que fazem sentido para uma classe de análise, devemos estudar cada caso de uso e escolher aquelas “coisas” que “pertencem” de forma razoável àquela classe. Além disso, a pergunta a seguir deve ser respondida para cada uma das classes: “Quais dados (compostos e/ou elementares) definem completamente esta classe no contexto do problema em mãos?”.

A título de ilustração, consideremos a classe **Sistema** definida para o *CasaSegura*. O proprietário de um imóvel pode configurar a função de segurança para que esta reflita informações sobre os sensores, tempo de resposta do alarme, ativação/desativação, de identificação e assim por diante. Podemos representar estes dados compostos da seguinte maneira:

informações de identificação = ID do sistema + número de telefone de verificação + estado do sistema

informações sobre a resposta do alarme = tempo de retardo + número de telefone

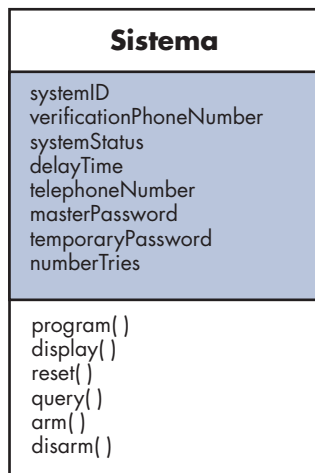
informações de ativação/desativação = senha-mestre + número de tentativas permitidas + senha temporária

Cada um dos dados à direita do sinal de igual poderia ser definido de forma mais extensiva a um nível elementar, porém, para nossos propósitos, eles constituem uma lista de atributos razoável para a classe **Sistema** (a parte sombreada da Figura 6.9).

Os sensores fazem parte do sistema global *CasaSegura* e ainda não estão listados como dados ou atributos na Figura 6.9. **Sensor** já foi definido como uma classe, e múltiplos objetos **Sensor** serão associados à classe **Sistema**. Em geral, evitamos definir um item como um atributo, caso mais de um dos itens deva ser associado à classe.

FIGURA 6.9

Diagrama de classes para a classe Sistema





Ao definir operações para uma classe de análise, concentre-se no comportamento orientado ao problema em vez de nos comportamentos necessários para a implementação.

6.5.3 Definição das operações

As operações definem o comportamento de um objeto. Embora existam muitos tipos diferentes de operações, em geral podem ser divididas em quatro grandes categorias: (1) operações que manipulam dados de alguma forma (por exemplo, adição, eliminação, reformatação, seleção), (2) operações que realizam um cálculo, (3) operações que pesquisam o estado de um objeto e (4) operações que monitoram um objeto em termos de ocorrências de um evento de controle. Tais funções são realizadas operando-se sobre atributos e/ou associações (Seção 6.5.5). Consequentemente, uma operação deve ter “conhecimento” da natureza dos atributos e associações das classes.

Como primeira iteração na obtenção de um conjunto de operações para uma classe de análise, podemos estudar novamente uma narrativa de processamento (ou caso de uso) e escolher aquelas operações que pertencem de forma razoável à classe. Para tanto, a análise sintática é mais uma vez estudada e os verbos são isolados. Alguns dos verbos serão as operações legítimas e podem ser facilmente associadas a uma classe específica. Por exemplo, da narrativa de processamento do *CasaSegura* apresentada anteriormente neste capítulo, veremos que “é atribuído um número e tipo aos sensores” ou “uma senha-mestre é programada para armar e desarmar o sistema”. Essas frases indicam uma série de coisas:

- Que uma operação *assign()* é relevante para a classe **Sensor**.
- Que uma operação *program()* será aplicada à classe **Sistema**.
- Que *arm()* e *disarm()* são operações que se aplicam à classe **Sistema**.

CASA SEGURA



Modelos de classes

Cena: Sala do Ed, quando se inicia o modelamento de requisitos.

Atores: Jamie, Vinod e Ed — todos os membros da equipe de engenharia de software do *CasaSegura*.

Conversa:

[Ed vem trabalhando na extração de classes do modelo de casos de uso para o AVC-EVC (apresentado em um quadro anterior deste capítulo) e está mostrando as classes que ele extraiu a seus colegas.]

Ed: Então, quando o proprietário de um imóvel quiser escolher uma câmera, ele terá de selecioná-la na planta da casa. Defini uma classe **Planta**. Eis o diagrama.

(Eles observam a Figura 6.10.)

Jamie: Portanto, **Planta** é um objeto que é colocado com paredes, portas, janelas e câmeras. É isso que estas linhas com identificação significam, certo?

Ed: Isso mesmo, elas são chamadas “associações”. Uma classe está associada a outra de acordo com as associações que eu mostrei. [As associações são discutidas na Seção 6.5.5.]

Vinod: Portanto, a planta real é feita de paredes e contém câmeras e sensores colocados nessas paredes. Como a planta da casa sabe onde colocar esses objetos?

Ed: Ela não sabe, mas as outras classes sim. Veja os atributos em, digamos, **TrechoParede**, que é usada para construir uma

parede. O trecho de parede tem coordenadas de início e fim e a operação *draw()* faz o resto.

Jamie: E o mesmo acontece com as janelas e portas. Parece-me que câmera possui alguns atributos extras.

Ed: Exatamente, preciso deles para fornecer informações sobre deslocamento e ampliação de imagens.

Vinod: Tenho uma pergunta. Por que a câmera possui um ID, mas os demais não? Percebi que você tem um atributo chamado **próximaParede**. Como **TrechoParede** saberá qual será a parede seguinte?

Ed: Boa pergunta, mas como dizem por aí, essa é uma decisão de projeto e, portanto, a postergarei até...

Jamie: Dá um tempo... Aposto que você já bolou isso.

Ed (sorrindo timidamente): É verdade, vou usar uma estrutura de listas que irei modelar quando chegarmos ao projeto. Se ficarmos muito presos à questão de separar análise e projeto, o nível de detalhe que tenho exatamente aqui poderia ser duvidoso.

Jamie: Para mim parece muito bom, mas tenho algumas outras perguntas.

(Jamie faz perguntas que resultam em pequenas modificações.)

Vinod: Você tem cartões CRC para cada um dos objetos? Em caso positivo, deveríamos simular seus papéis, apenas para ter certeza de que nada tenha escapado.

Ed: Não estou bem certo de como fazê-lo.

Vinod: Não é difícil e realmente vale a pena. Vou lhes mostrar.

Após uma investigação mais apurada, é provável que a operação *program()* seja dividida em uma série de suboperações mais específicas exigidas para configurar o sistema. Por exemplo, *program()* implica a especificação de números de telefone, a configuração de características do sistema (por exemplo, criar a tabela de sensores, introduzir características do alarme) e a introdução de senha(s). Mas, por enquanto, especificaremos *program()* como uma única operação.

Além da análise sintática, pode-se ter uma melhor visão sobre outras operações considerando-se a comunicação que ocorre entre os objetos. Os objetos se comunicam passando mensagens entre si. Antes de prosseguirmos com a especificação de operações, exploraremos essa questão em mais detalhes.

6.5.4 Modelagem CRC (Classe-Responsabilidade-Colaborador)

A modelagem CRC (*classe-responsabilidade-colaborador*) [Wir90] fornece uma maneira simples para identificar e organizar as classes que são relevantes para os requisitos do sistema ou produto. Ambler [Amb95] descreve a modelagem CRC da seguinte maneira:

Um modelo CRC é, na verdade, um conjunto de fichas-padrão que representam classes. Os cartões são divididos em três seções. Ao longo da parte superior do cartão escrevemos o nome da classe. No corpo do cartão enumeramos as responsabilidades da classe do lado esquerdo e os colaboradores do lado direito.

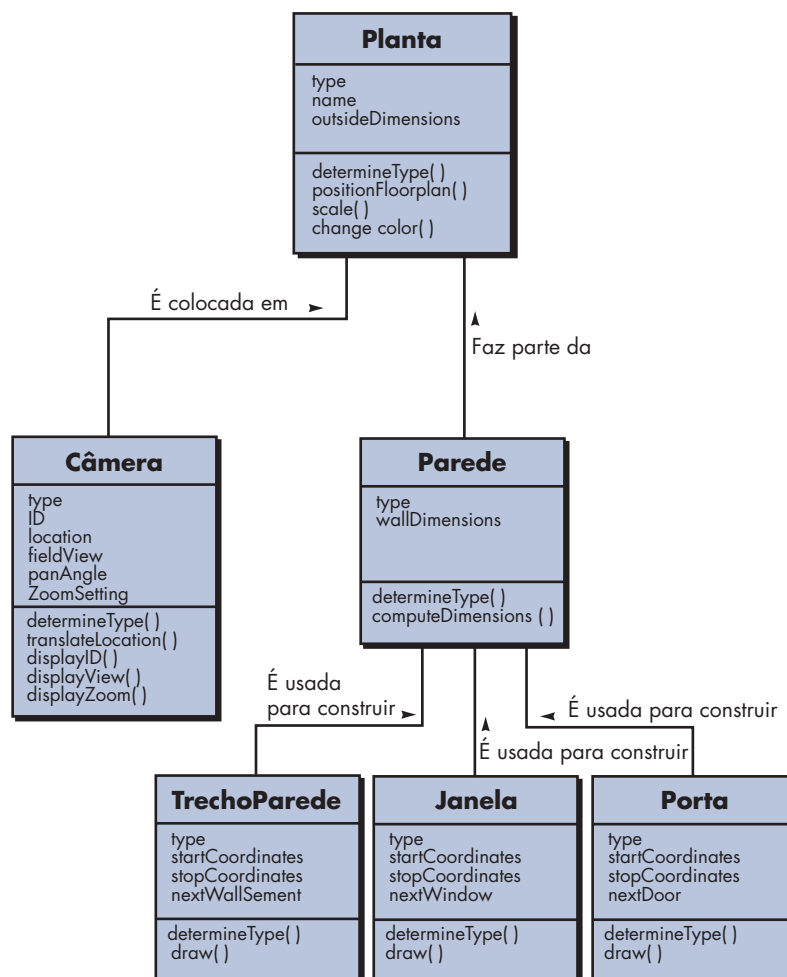
Na realidade, o modelo CRC pode fazer uso de fichas reais ou virtuais. O intuito é desenvolver uma representação organizada das classes. As *responsabilidades* são os atributos e as

“Um dos propósitos dos cartões CRC é fazer com que as falhas apareçam logo, com frequência e de forma barata. É muito mais barato rasgar um monte de cartões do que reorganizar uma grande quantidade de código-fonte.”

C. Horstmann

FIGURA 6.10

Diagrama de classes para Planta (veja o quadro de discussão)



operações que são relevantes para a classe. Em outras palavras, responsabilidade é “qualquer coisa que a classe sabe ou faz” [Amb95]. *Colaboradores* são aquelas classes que são necessárias para fornecer a uma classe as informações necessárias para completar uma responsabilidade. Em geral, *colaboração* implica uma solicitação de informações ou uma solicitação de alguma ação.

Um cartão CRC simples para a classe **Planta** é ilustrado na Figura 6.11. A lista de responsabilidades mostrada no cartão CRC é preliminar e sujeita a acréscimos ou modificações. As classes **Parede** e **Câmera** são indicadas ao lado da responsabilidade que irá requerer sua colaboração.

WebRef

Uma excelente discussão sobre esses tipos de classes pode ser encontrada em www.theumlcafe.com/a0079.htm.

“Os objetos podem ser classificados cientificamente em três grandes categorias: aqueles que não funcionam, aqueles que quebram e aqueles que se perdem.”

Russell Baker

Classes. Foram apresentadas diretrizes básicas para a identificação de classes e objetos no início deste capítulo. A taxonomia dos tipos de classe da Seção 6.5.1 pode ser estendida considerando-se as seguintes categorias:

- *Classes de entidades*, também chamadas *classes de modelo* ou *de negócio*, são extraídas diretamente do enunciado do problema (por exemplo, **Planta** e **Sensor**). Estas representam, tipicamente, coisas que devem ser armazenadas em um banco de dados e persistem ao longo de toda a existência da aplicação (a menos que sejam especificamente eliminadas).
- *Classes de fronteira* são usadas para criar a interface (por exemplo, tela interativa ou relatórios impressos) que o usuário vê e interage à medida que o software é usado. Os objetos de entidades contêm informações importantes para os usuários, mas eles não são exibidos. As *classes de fronteira* são desenvolvidas com a responsabilidade de gerenciar a forma através da qual os objetos de entidades são representados para os usuários. Por exemplo, uma classe de fronteira chamada **JanelaCâmera** teria a responsabilidade de exibir imagens de câmeras de vigilância do sistema *CasaSegura*.
- *Classes de controle* gerenciam uma “unidade de trabalho” [UML03] do início ao fim. Isto é, as classes de controle podem ser desenvolvidas para gerenciar: (1) a criação ou a atualização de objetos de entidades, (2) a instanciação de objetos de fronteira à medida que forem obtendo informações dos objetos de entidades, (3) comunicação complexa entre conjuntos de objetos, (4) validação de dados transmitidos entre objetos ou entre o usuário e a aplicação. Em geral, as classes de controle não são consideradas até que a atividade de projeto tenha sido iniciada.

FIGURA 6.11

Um modelo de cartão CRC

Classe: Planta	
Descrição	
Responsabilidade:	Colaborador:
Incorpora paredes, portas e janelas	
Mostra a posição das câmeras de vídeo	
Define o nome/tipo da planta	
Gerencia o posicionamento na planta	
Amplia/reduz a planta para exibição	
Incorpora paredes, portas e janelas	Parede
Mostra a posição das câmeras de vídeo	Câmera

 Quais diretrizes podem ser aplicadas para alocar responsabilidades às classes?

Responsabilidades. Diretrizes básicas para a identificação das responsabilidades (atributos e operações) foram apresentadas nas Seções 6.5.2 e 6.5.3. Wirfs-Brock e seus colegas [Wir90] sugerem cinco diretrizes para alocação de responsabilidades às classes:

1. A inteligência do sistema deve ser distribuída pelas classes para melhor atender às necessidades do problema. Toda aplicação engloba certo grau de inteligência; aquilo que o sistema sabe e aquilo que ele pode fazer. Essa inteligência pode ser distribuída pelas classes em uma série de maneiras. Classes “burras” (aquelas com poucas responsabilidades) podem ser modeladas para atuar como serventes para algumas poucas classes “inteligentes” (aquelas com muitas responsabilidades). Embora essa abordagem faça com que o fluxo de controle em um sistema se torne simples, ela apresenta algumas desvantagens: concentra toda a inteligência em algumas poucas classes, tornando as mudanças mais difíceis e tende a exigir mais classes e, portanto, um esforço de desenvolvimento maior.

Se a inteligência do sistema for distribuída de forma mais homogênea pelas classes de uma aplicação, cada objeto conhecerá e fará apenas algumas poucas coisas (que em geral são bem focadas), a coesão do sistema aumentará.¹⁷ Isso aumenta a facilidade de manutenção do software e reduz o impacto dos efeitos colaterais devido a mudanças.

Para determinar se a inteligência do sistema está distribuída de maneira apropriada, as responsabilidades indicadas em cada cartão CRC modelo devem ser avaliadas para determinar se alguma classe tem uma lista extraordinariamente longa de responsabilidades. Isso indica uma concentração de inteligência.¹⁸ Além disso, as responsabilidades para cada classe deveriam exibir o mesmo nível de abstração. Por exemplo, entre as operações listadas para uma classe agregada denominada **ContaCorrente** um revisor indica duas responsabilidades: *verificar-saldo-da-conta* e *dar-baixa-cheques-compensados*. A primeira operação (responsabilidade) implica um complexo procedimento lógico e matemático. A segunda é uma atividade administrativa simples. Como essas duas operações não se encontram no mesmo nível de abstração, *dar-baixa-cheques-compensados* deve ser colocada dentro das responsabilidades de **EntradaCheques**, uma classe que é englobada pela classe agregada **ContaCorrente**.

- 2. Cada responsabilidade deve ser declarada da forma mais genérica possível.** Essa diretriz implica que as responsabilidades gerais (tanto atributos quanto operações) devem estar no topo da hierarquia de classes (por elas serem genéricas, serão aplicáveis a todas as subclasses).
- 3. As informações e o comportamento relativos a elas devem residir na mesma classe.** Isso atende o princípio da orientação a objetos denominado *encapsulamento*. Os dados e os processos que manipulam os dados devem ser empacotados como uma unidade coesiva.
- 4. As informações sobre um item devem estar em uma única classe e não distribuída por várias classes.** Uma única classe deve assumir a responsabilidade pelo armazenamento e manipulação de um tipo de informação específico. Tal responsabilidade não deve, em geral, ser compartilhada por uma série de classes. Se as informações forem distribuídas, o software se torna mais difícil de ser mantido e testado.
- 5. Quando apropriado, as responsabilidades devem ser compartilhadas entre classes relacionadas.** Há muitos casos em que uma série de objetos relacionados deve apresentar o mesmo comportamento ao mesmo tempo. Como exemplo, consideremos um videogame que precise exibir as seguintes classes: **Jogador**, **CorpoJogador**, **BraçosJogador**, **PernasJogador**, **CabeçaJogador**. Cada uma dessas classes possui seus próprios atributos (por exemplo,

¹⁷ Coesão é um conceito de projeto que será discutido no Capítulo 8.

¹⁸ Em tais casos, talvez seja necessário subdividir a classe em várias classes ou subsistemas completos para distribuir a inteligência de forma mais eficaz.

posição, orientação, cor, velocidade) e todas têm de ser atualizadas e exibidas à medida que o usuário manipula um *joystick*. Consequentemente, as responsabilidades *atualizar()* e *exibir()* devem ser compartilhadas por cada um dos objetos citados. **Jogador** sabe quando algo mudou e *atualizar()* se faz necessária. Ela colabora com os demais objetos para atingir uma nova posição ou orientação, porém cada objeto controla sua própria visualização.

Colaborações. As classes cumprem suas responsabilidades de duas formas: (1) Uma classe pode usar suas próprias operações para manipular seus próprios atributos, cumprindo, portanto, determinada responsabilidade ou (2) uma classe pode colaborar com outras classes. Wirfs-Brock e seus colegas [Wir90] definem as colaborações da seguinte forma:

As colaborações representam solicitações de um cliente a um servidor no cumprimento de uma responsabilidade do cliente. A colaboração é a expressão do contrato entre o cliente e o servidor... Dizemos que um objeto colabora com outro objeto se, para cumprir uma responsabilidade, ele precisar enviar ao outro objeto quaisquer mensagens. Uma colaboração simples flui em uma direção — representando uma solicitação do cliente ao servidor. Do ponto de vista do cliente, cada uma de suas colaborações está associada a determinada responsabilidade implementada pelo servidor.

As colaborações são identificadas determinando se uma classe pode ou não cumprir cada responsabilidade por si só. Caso não possa, ela precisa interagir com outra classe. Daí, a colaboração.

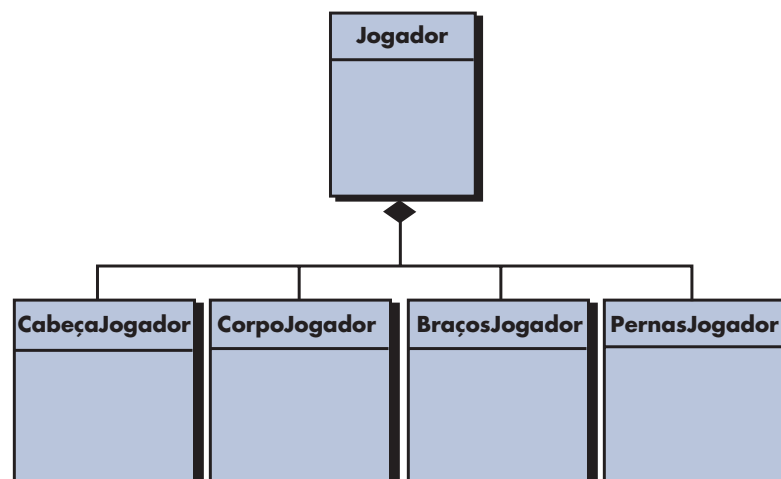
Como exemplo, consideremos a *função de segurança domiciliar do CasaSegura*. Como parte do procedimento de ativação, o objeto **PaineldeControle** deve determinar se algum sensor está aberto. A responsabilidade chamada *determinar-estado-sensor()* é definida. Se existirem sensores abertos, **PaineldeControle** tem de ativar um atributo de estado para “não preparado”. As informações dos sensores podem ser adquiridas de cada objeto **Sensor**. Consequentemente, a responsabilidade *determinar-estado-sensor()* pode ser cumprida apenas se **PaineldeControle** trabalhar em colaboração com **Sensor**.

Para ajudarmos na identificação dos colaboradores, podemos examinar três relacionamentos genéricos diferentes entre as classes [Wir90]: (1) o relacionamento *é-parte-de*, (2) o relacionamento *tem-conhecimento-de* e (3) o relacionamento *depende-de*. Cada um desses três relacionamentos genéricos é considerado remidamente nos parágrafos a seguir.

Todas as classes que fazem parte de uma classe agregada são interligadas à classe agregada por meio de um relacionamento *é-parte-de*. Considerando as classes definidas para o videogame citado anteriormente, a classe **CorpoJogador** *faz-parte-do* **Jogador**, **assim como** **BraçosJogador**, **PernasJogador** e **CabeçaJogador**. Em UML, tais relacionamentos são representados na forma da agregação mostrada na Figura 6.12.

FIGURA 6.12

Uma classe composta de agregação



Quando uma classe tem de adquirir informações de outra classe, é estabelecido o relacionamento *tem-conhecimento-de*. A responsabilidade *determinar-estado-sensor()* citada anteriormente é um exemplo de um relacionamento *tem-conhecimento-de*.

A relação *depende-de* implica que duas classes têm uma dependência que não é conseguida por *tem-conhecimento-de* ou *é-parte-de*. Por exemplo, **CabeçaJogador** sempre tem de estar interligado ao **CorpoJogador** (a menos que o videogame seja particularmente violento), embora cada objeto pudesse existir sem o conhecimento direto do outro. Um atributo do objeto **CabeçaJogador** chamado **posição-central** é determinado a partir da posição central do **CorpoJogador**. Essa informação é obtida através de um terceiro objeto, **Jogador**, que a adquire de **CorpoJogador**. Portanto, **CabeçaJogador** *depende-do* **CorpoJogador**.

Em todos os casos, o nome da classe colaboradora é registrado no cartão CRC modelo, próximo à responsabilidade que gerou a colaboração. Consequentemente, o cartão contém uma lista de responsabilidades e as colaborações correspondentes que permitem que as responsabilidades sejam cumpridas (Figura 6.11).

Quando um modelo CRC completo tiver sido desenvolvido, os interessados poderão revisar o modelo usando a seguinte abordagem [Amb95]:

1. Todos os participantes da revisão (do modelo CRC) recebem um subconjunto dos cartões CRC. Os cartões que colaboram devem ser separados (nenhum revisor deve ter dois cartões que colaboram).
2. Todos os cenários de uso (e diagramas de casos de uso correspondentes) devem ser organizados em categorias.
3. O líder da revisão lê o caso de uso pausadamente. À medida que o líder da revisão chega a um objeto com nome, ele passa uma ficha para a pessoa que está com o cartão da classe correspondente. Por exemplo, um caso de uso para o *CasaSegura* conteria a seguinte narrativa:

O proprietário do imóvel observa o painel de controle do *CasaSegura* para determinar se o sistema está pronto para operar. Se o sistema não estiver pronto, o proprietário do imóvel deve fechar manualmente as janelas/portas de modo que o indicador pronto esteja presente. [Um indicador não preparado implica que um sensor está aberto, isto é, que uma porta ou janela está aberta.]

Quando o líder da revisão chega em “painel de controle”, na narrativa de caso de uso, a ficha é passada para a pessoa que está com o cartão **PaineldeControle**. A frase “implica que um sensor está aberto” requer que o cartão contenha a responsabilidade que irá validar esta implicação (a responsabilidade *determinar-estado-sensor()* realiza isso). Próximo à responsabilidade no cartão se encontra o colaborador **Sensor**. A ficha é então passada para o objeto **Sensor**.

4. Quando a ficha é passada, solicita-se ao portador do cartão **Sensor** que descreva as responsabilidades anotadas no cartão. O grupo determina se uma (ou mais) das responsabilidades satisfaz à necessidade do caso de uso.
5. Se as responsabilidades e colaborações anotadas nos cartões não puderem satisfazer ao caso de uso, as modificações são feitas nos cartões. Estas podem incluir a definição das novas classes (e os cartões CRC correspondentes) ou a especificação das responsabilidades novas ou revisadas ou das colaborações em cartões existentes.

Esse *modus operandi* prossegue até que o caso de uso seja finalizado. Quando todos os casos de uso tiverem sido revistos, a modelagem de requisitos continua.

CASA SEGURA



Modelos CRC

Cena: Sala do Ed, quando se inicia a modelagem de requisitos.

Atores: Jamie, Vinod e Ed — todos os membros da equipe de engenharia de software do *CasaSegura*.

Conversa:

[Vinod decidiu mostrar a Ed como desenvolver cartões CRC dando-lhe um exemplo.]

Vinod: Enquanto você trabalhava na função de vigilância e Jamie estava envolvido com a função de segurança, trabalhei na função de administração domiciliar.

Ed: E em que ponto você se encontra? O Marketing vive mudando de ideia.

Vinod: Eis o caso de uso preliminar para toda a função... Nós o refinamos um pouco, mas isso deve lhe dar uma visão geral...

Caso de uso: A função de administração domiciliar do *CasaSegura*.

Narrativa: Queremos usar a interface de administração domiciliar em um PC ou via Internet para controlar dispositivos eletrônicos que possuem controladores de interface sem fio. O sistema deve permitir que eu ligue e desligue luzes específicas, controle aparelhos que estiverem conectados a uma interface sem fio, configure o meu sistema de aquecimento e ar-condicionado para as temperaturas que eu desejar. Para tanto, quero escolher os dispositivos com base na planta da casa. Cada dispositivo tem de ser identificado na planta da casa. Como recurso opcional, quero controlar todos os dispositivos audiovisuais — áudio, televisão, DVD, gravadores digitais e assim por diante. Com uma única seleção, quero ser capaz de configurar a casa inteira para várias situações. Uma delas é *em casa*, a outra é *fora de casa*, a terceira é *viagem de fim de semana* e a quarta é *viagem prolongada*. Todas essas situações terão ajustes de configuração aplicados a todos os dispositivos. Nos estados *viagem de fim de semana* e *viagem prolongada*, o sistema deve acender e apagar as luzes da casa em intervalos aleatórios (para parecer que alguém está em casa) e controlar o sistema de aquecimento e ar-condicionado. Devo ser capaz de cancelar essas configurações via Internet com a proteção de uma senha apropriada...

Ed: O pessoal do hardware já bolou todas as interfaces sem fio?

Vinod (sorrindo): Eles estão trabalhando nisso; digamos que não há nenhum problema. De qualquer forma, extraí um monte de classes para a administração domiciliar e podemos usar uma delas como exemplo.

Usemos a classe **InterfaceAdministraçãoDomiciliar**.

Ed: OK... Portanto, as responsabilidades são... Os atributos e as operações para a classe, e as colaborações são as classes para as quais as responsabilidades apontam.

Vinod: Pensei que você não entendesse sobre CRC.

Ed: Um pouquinho, mas vá em frente.

Vinod: Portanto, eis minha definição de classe para **InterfaceAdministraçãoDomiciliar**.

Atributos:

optionsPanel — contém informações sobre os botões que permitem ao usuário escolher a funcionalidade.

situationPanel — contém informações sobre os botões que permitem ao usuário escolher a situação.

floorplan — o mesmo que o objeto de vigilância, porém este aqui mostra os dispositivos.

deviceIcons — informações sobre os ícones representando luzes, aparelhos, HVAC etc.

devicePanels — simulação do painel de controle de dispositivos ou de aparelhos; possibilita o controle.

Operações:

displayControl(), selectControl(), displaySituation(), select situation(), accessFloorplan(), selectDeviceIcon(), displayDevicePanel(), accessDevicePanel()...

Classe: InterfaceAdministraçãoDomiciliar

Responsabilidade

displayControl()
selectControl()
displaySituation()
selectSituation()
accessFloorplan()
...

Colaborador

PaineldeOpção (classe)
PaineldeOpção (classe)
PaineldeSituação (classe)
PaineldeSituação (classe)
Planta (classe)...

Ed: Portanto, quando a operação *accessFloorplan()* for chamada, ela colaborará com o objeto **FloorPlan** exatamente com aquela que desenvolvemos para a função de vigilância. Espere, tenho uma descrição dela aqui comigo. (Eles observam a Figura 6.10.)

Vinod: Exatamente. E se quiséssemos rever o modelo de classes inteiro, poderíamos começar com este cartão, depois ir para o cartão do colaborador e a partir deste ponto para um dos colaboradores do colaborador, e assim por diante.

Ed: Uma boa forma de descobrir omissões ou erros.

Vinod: Isso aí.

6.5.5 Associações e dependências

Em muitos casos, duas classes de análise são relacionadas entre si de alguma maneira, de modo muito parecido como dois objetos de dados poderiam estar relacionados entre si (Seção 6.4.3). Na UML essas relações são chamadas *associações*. Referindo-se novamente à Figura 6.10, a classe **Planta** é definida identificando-se um conjunto de associações entre **Planta** e duas outras classes, **Câmera** e **Parede**. A classe **Parede** é associada a três classes que permitem que uma parede seja construída, **TrechoParede**, **Janela** e **Porta**.

PONTO-CHAVE

A associação define um relacionamento entre classes. A multiplicidade define quantos de uma classe estão relacionados com quantos de uma outra classe.

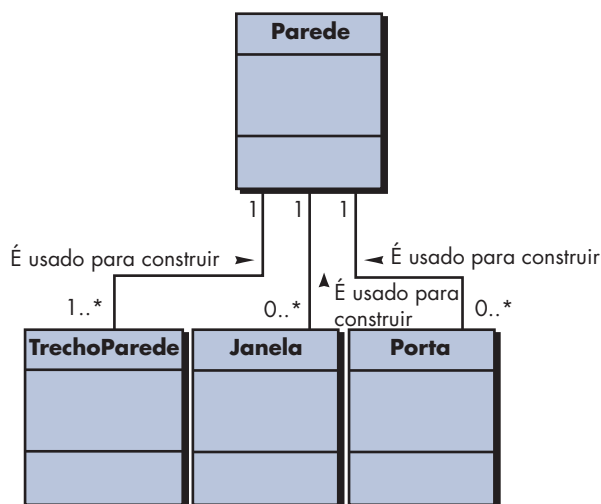
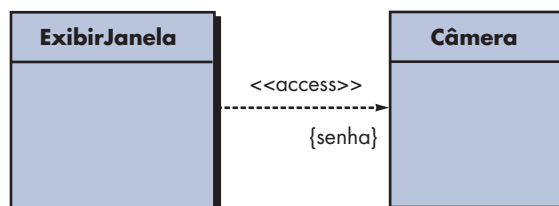


O que é um estereótipo?

Em alguns casos, uma associação pode ser mais bem definida indicando-se a *multiplicidade*. Referindo-se à Figura 6.10, um objeto **Parede** é construído por meio de um ou mais objetos **TrechoParede**. Além disso, o objeto **Parede** pode conter nenhum ou alguns objetos **Janela** e nenhum ou alguns objetos **Porta**. Essas restrições de multiplicidade são ilustradas na Figura 6.13, em que “um ou mais” é representado usando-se 1..* e “zero ou mais” por 0..*. Na UML, o asterisco indica um limite superior infinito no intervalo.¹⁹

Em muitos casos, existe um relacionamento cliente/servidor entre duas classes de análise. Em tais casos, uma classe cliente depende da classe servidora de alguma forma e se estabelece um relacionamento *relação de dependência*. Dependências são definidas por um estereótipo. *Estereótipo* é um “mecanismo de extensibilidade” [Ar102] dentro da UML que nos permite definir um elemento de modelagem especial cuja semântica é definida de forma personalizada. Na UML os estereótipos são representados entre << >> (por exemplo, <<stereotype>>).

Como exemplo de uma dependência simples no sistema de vigilância *CasaSegura*, um objeto **Câmera** (neste caso, a classe servidora) fornece uma imagem de vídeo para um objeto **ExibirJanela** (neste caso, a classe cliente). O relacionamento entre esses dois objetos não é uma associação simples, embora não exista uma associação de dependência. Em um caso de uso escrito para vigilância (não mostrado), você toma conhecimento de que uma senha especial tem de ser fornecida para visualizar posições de câmera específicas. Uma maneira de conseguir isso é fazer com que **Câmera** solicite uma senha e então dê permissão para **ExibirJanela** produzir a exibição de vídeo. Isso pode ser representado como mostra a Figura 6.14 em que <<access>> implica que o uso da saída de câmera é controlado por uma senha especial.

FIGURA 6.13**Multiplicidade****FIGURA 6.14****Dependências**

¹⁹ Outros relacionamentos de multiplicidade — um para um, um para vários, vários para vários, um para um intervalo especificado com limites inferior e superior e outros — podem ser indicadas como parte de uma associação.

PONTO-CHAVE

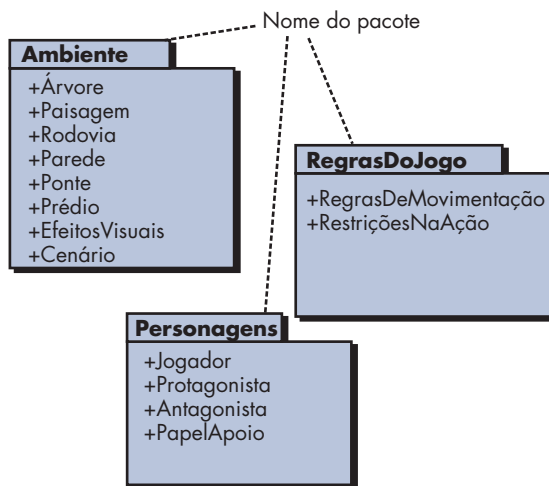
Um pacote é usado para montar um conjunto de classes relacionadas.

6.5.6 Pacotes de análise

Uma importante parte da modelagem de análise é a categorização. Vários elementos do modelo de análise (por exemplo, casos de uso, classes de análise) são categorizados de modo que são empacotados como um agrupamento — denominado *pacote de análise* — que recebe um nome representativo.

Para ilustrarmos o uso de pacotes de análise, consideremos o videogame introduzido anteriormente. À medida que o modelo de análise para o videogame é desenvolvido, um número maior de classes é extraído. Algumas focalizam o ambiente do jogo — as cenas que o usuário vê à medida que o jogo vai se desenrolando. Classes como **Árvore**, **Paisagem**, **Rodovia**, **Parede**, **Ponte**, **Prédio** e **EfeitoVisual** poderiam cair dentro desta categoria. Outras focalizam os personagens do jogo, descrevendo suas características físicas, ações e restrições. Classes como **Jogador** (descrito anteriormente), **Protagonista**, **Antagonista** e **PapéisApoio** poderiam ser definidas. Outras ainda descrevem as regras do jogo — como um jogador navega pelo ambiente. Classes como **RegrasDeMovimentação** e **RestriçõesNaAção** são candidatas aqui. Poderiam existir muitas outras categorias. Essas classes podem ser agrupadas em pacotes de análise conforme mostra a Figura 6.15.

O sinal de mais antes do nome da classe de análise em cada pacote indica que as classes têm visibilidade pública e são, conseqüentemente, acessíveis de outros pacotes. Embora não sejam mostrados na figura, outros símbolos podem anteceder um elemento dentro de um pacote. Um sinal de menos indica que um elemento está oculto de todos os demais pacotes e um símbolo # indica que um elemento é acessível apenas para pacotes contidos em determinado pacote.

FIGURA 6.15**Pacotes****6.6 RESUMO**

O objetivo da modelagem de requisitos é criar uma variedade de representações que descrevem aquilo que o cliente requer, estabelece uma base para a criação de um projeto de software e define um conjunto de requisitos que podem ser validados assim que o software for construído. O modelo de requisitos preenche a lacuna entre uma representação sistêmica que descreve o sistema como um todo ou a funcionalidade de negócio e um projeto de software que descreve a arquitetura da aplicação de software, a interface do usuário e a estrutura no nível de componentes.

Os modelos baseados em cenário representam requisitos de software sob o ponto de vista do usuário. O caso de uso — uma narrativa ou descrição dirigida por modelos de uma interação entre um ator e o software — é o principal elemento da modelagem. Obtido durante o levantamento de requisitos, o caso de uso define as etapas fundamentais para uma função ou interação específica. O grau de formalidade dos casos de uso e detalhes varia, porém o resultado final fornece a entrada necessária para as demais atividades de modelagem de análise. Os cenários também podem ser descritos usando-se um diagrama de atividades — uma representação gráfica do tipo fluxograma que represente o fluxo de processamento dentro de um cenário específico. Os diagramas de raia ilustram como o fluxo de processamento é alocado a vários atores ou classes.

A modelagem de dados é usada para descrever o espaço de informações que serão construídas ou manipuladas pelo software. A modelagem de dados começa pela representação dos objetos de dados — informações compostas que devem ser compreendidas pelo software. Os atributos de cada objeto de dados são identificados e os relacionamentos entre objetos de dados, descritos.

A modelagem baseada em classes usa informações extraídas dos elementos de modelagem de dados baseados em cenários para identificar as classes de análise. Uma análise sintática pode ser usada para extrair classes, atributos e operações candidatos com base em narrativas textuais. São definidos os critérios para a definição de uma classe. Um conjunto de cartões classe-responsabilidade-colaborador pode ser usado para definir relações entre classes. Além disso, uma variedade de notação da modelagem UML pode ser aplicada para definir hierarquias, relacionamentos, associações, agregações e dependências entre as classes. Pacotes de análise são usados para categorizar e agrupar classes de maneira que as tornem mais administráveis para grandes sistemas.

PROBLEMAS E PONTOS A PONDERAR

- 6.1. Existe a possibilidade de começar a codificar logo depois de um modelo de análise ter sido criado? Justifique sua resposta e, em seguida, argumente ao contrário.
- 6.2. Uma regra prática para análise é que o modelo “deve se concentrar nos requisitos visíveis dentro do domínio do negócio ou problema”. Que tipos de requisitos *não* são visíveis nesses domínios? Forneça alguns exemplos.
- 6.3. Qual o propósito da análise de domínio? Como está relacionada com o conceito de padrões de requisitos?
- 6.4. É possível desenvolver um modelo de análise efetivo sem desenvolver todos os quatro elementos da Figura 6.3? Explique.
- 6.5. Foi-lhe solicitado construir um dos seguintes sistemas:
 - a. um sistema de matrícula em cursos baseado em rede para a sua universidade.
 - b. um sistema de processamento de pedidos baseado na Web para uma loja de informática.
 - c. um sistema de faturas simples para um pequeno negócio.
 - d. um livro de receitas baseado na Internet que é embutido em forno elétrico ou micro-ondas.

Escolha o sistema de seu interesse e desenvolva um diagrama entidade-relação que descreva objetos de dados, relacionamentos e atributos.

- 6.6. O departamento de obras públicas de uma grande cidade decidiu desenvolver um sistema de tapa-buracos (*pothole tracking and repair system*, PHTRS) baseado na Web. Segue uma descrição:

Os cidadãos podem entrar em um site e relatar o local e a gravidade dos buracos. À medida que são relatados, os buracos são registrados em um “sistema de reparos do departamento de obras públicas” e recebem um número identificador, armazenado pelo endereço (nome da rua), tamanho (em

uma escala de 1 a 10), localização (no meio da rua, meio-fio etc.), bairro (determinado com base no endereço) e prioridade para o reparo (determinada segundo o tamanho do buraco). Os dados de solicitação de trabalho são associados a cada buraco e incluem a localização e o tamanho do buraco, equipe de obras identificando o número, o número de operários da equipe, equipamento alocado, horas usadas para o reparo, estado do buraco (trabalho em andamento, reparado, reparo temporário, não reparado), quantidade de material de preenchimento utilizado e custo do reparo (calculado com base nas horas utilizadas, no número de pessoas, no material e no equipamento usado). Por fim, é criado um arquivo de danos para armazenar informações sobre o dano relatado devido ao buraco e que inclui o nome, endereço e telefone do cidadão, tipo de dano e custo monetário do dano. O PHTRS é um sistema on-line; todas as consultas devem ser feitas interativamente.

- a. Desenhe um diagrama de caso de uso UML para o sistema PHTRS. Você terá de fazer uma série de suposições sobre a maneira através da qual um usuário interage com esse sistema.
 - b. Desenvolva um modelo de classes para o sistema PHTRS.
- 6.7.** Escreva um caso de uso baseado em modelo para o sistema de administração domiciliar *CasaSegura* descrito informalmente no quadro após a Seção 6.5.4.
- 6.8.** Desenvolva um conjunto completo de cartões CRC sobre o produto ou sistema que você escolher como parte do Problema 6.5.
- 6.9.** Realize uma revisão dos cartões CRC com seus colegas. Quantas classes, responsabilidades e colaboradores adicionais são acrescentados como consequência da revisão?
- 6.10.** O que é um pacote de análise e como poderia ser usado?

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Os casos de uso podem servir como base para todas as abordagens de modelagem de requisitos. O tema é discutido de forma abrangente por Rosenberg e Stephens (*Use Case Driven Object Modeling with UML: Theory and Practice*, Apress, 2007), Denny (*Succeeding with Use Cases: Working Smart to Deliver Quality*, Addison-Wesley, 2005), Alexander e Maiden (eds.) (*Scenarios, Stories, Use Cases: Through the Systems Development Life-Cycle*, Wiley, 2004), Bittner e Spence (*Use Case Modeling*, Addison-Wesley, 2002), Cockburn [Coc01b] e outras referências citadas nos Capítulos 5 e 6.

A modelagem de dados apresenta um útil método para examinar o espaço de informações. Livros como os de Hoberman [Hob06] e Simsion e Witt [Sim05] são tratados relativamente completos. Além disso, Allen e Terry (*Beginning Relational Data Modeling*, 2. ed., Apress, 2005), Allen (*Data Modeling for Everyone*, Wrox Press, 2002), Teorey e seus colegas (*Database Modeling and Design: Logical Design*, 4. ed., Morgan Kaufmann, 2005) e Carlis e Maguire (*Mastering Data Modeling*, Addison-Wesley, 2000) trazem tutoriais detalhados para criação de modelos de dados de alta qualidade. Um interessante livro de Hay (*Data Modeling Patterns*, Dorset House, 1995) lista padrões de modelos de dados típicos encontrados em diversas áreas de aplicação.

As técnicas de modelagem UML que podem ser aplicadas tanto na análise quanto no projeto são discutidos por O'Docherty (*Object-Oriented Analysis and Design: Understanding System Development with UML 2.0*, Wiley, 2005), Arlow e Neustadt (*UML 2 and the Unified Process*, 2. ed., Addison-Wesley, 2005), Roques (*UML in Practice*, Wiley, 2004), Dennis e seus colegas (*Systems Analysis and Design with UML Version 2.0*, Wiley, 2004), Larman (*Applying UML and Patterns*, 2. ed., Prentice-Hall, 2001) e Rosenberg e Scott (*Use Case Driven Object Modeling with UML*, Addison-Wesley, 1999).

Uma ampla gama de fontes de informação sobre modelagem de requisitos se encontra à disposição na Internet. Uma lista atualizada de referências na Web, relevantes à modelagem de requisitos, pode ser encontrada no site www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm.